

UMCS

UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE
Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Informatyka**
Specjalność: **–**

Igor Jarosław Mazur
nr albumu: 290989

Aplikacja internetowa do projektowania struktury nici DNA w celu zapisu informacji

A web-based application for designing DNA strand structure to store information

Praca licencjacka
pod kierunkiem dr Marka Miśkiewicza

Lublin 2025

Spis treści

Wstęp **4**

1 Kodowanie informacji w niciach DNA **5**
1.1 Kwas deoksyrybonukleinowy (DNA) 5
1.2 Zastosowania syntetycznego DNA 10
1.3 Zapis informacji w DNA 12
1.4 Kryptografia w DNA 14
1.4.1 Tradycyjne metody kryptografii 14
1.4.2 Kryptografia oparta na DNA 15

2 Aplikacja do zarządzania danymi zapisanych w DNA **16**
2.1 Opis problemu 17
2.2 Mechanizm kodowania danych przez użytkownika w DNA 18
2.3 Opis aplikacji webowej 21
2.4 Użyte narzędzia 22
2.4.1 Python 22
2.4.2 Django REST Framework 22
2.4.3 PostgreSQL 24
2.4.4 JavaScript 25
2.4.5 React 25
2.4.6 Postman 25

3 Implementacja aplikacji webowej **26**
3.1 Przygotowanie środowiska do pracy 27
3.2 Programowanie warstwy backendowej 29
3.3 Testowanie interfejsu API 32
3.4 Tworzenie warstwy frontendowej 38
3.5 Aspekty bezpieczeństwa aplikacji 50

Podsumowanie **52**

Spis rysunków **54**

Spis listingów **55**

Bibliografia **57**

Wstęp

Dynamiczny rozwój technologii cyfrowych generuje bezprecedensowy wzrost ilości danych, co stawia przed informatyką wyzwanie poszukiwania nowych, bardziej wydajnych nośników informacji. W tym kontekście coraz większe zainteresowanie budzi kwas deoksyrybonukleinowy (DNA), który dzięki swojej niezwykłej gęstości zapisu i trwałości, stanowi obiecującą alternatywę dla tradycyjnych metod przechowywania danych.

Pomimo rosnącego potencjału, narzędzia umożliwiające praktyczne wykorzystanie algorytmów kodowania informacji w DNA są często ograniczone do postaci skryptów badawczych, co stanowi barierę dla szerszego grona użytkowników. W odpowiedzi na te wyzwania, celem niniejszej pracy jest zaprojektowanie, implementacja i opis aplikacji webowej, która pełni rolę funkcjonalnego interfejsu dla istniejącego algorytmu konwersji danych na sekwencje nukleotydowe.

Architektura aplikacji została oparta o REST API, co zapewnia skalowalność oraz możliwość obsługi wielu użytkowników i zarządzanie przypisanymi do nich danymi. Rozwiązanie to demonstruje praktyczne połączenie koncepcji z dziedziny biologii molekularnej z technologiami tworzenia oprogramowania.

Należy jednak podkreślić, że zrealizowany projekt ma charakter prototypu (ang. proof of concept). Jego głównym zadaniem jest weryfikacja koncepcji oraz prezentacja umiejętności w zakresie integracji technologii backendowych i frontendowych. Wdrożono podstawowe mechanizmy bezpieczeństwa, takie jak uwierzytelnianie oparte na tokenach JWT oraz haszowanie haseł z użyciem PBKDF2. Aplikacja celowo pomija jednak zaawansowane środki ochrony, co oznacza, że w obecnej formie nie jest przeznaczona do przetwarzania wrażliwych informacji ani do wdrożenia w środowisku produkcyjnym.

4:5086660669

Rozdział 1

Kodowanie informacji w niciach DNA

1.1 Kwas deoksyrybonukleinowy (DNA)

Dynamiczny rozwój biologii molekularnej oraz technologii informatycznych sprawił, że kwas deoksyrybonukleinowy (DNA) zyskał nowe, nietypowe zastosowania wykraczające poza obszar nauk przyrodniczych. Współcześnie dostępne metody syntezy i sekwencjonowania DNA umożliwiają nie tylko jego analizę, lecz także kontrolowaną modyfikację, co otwiera drogę do zastosowania tego nośnika w takich dziedzinach jak inżynieria danych, kryptografia czy trwałe archiwizowanie informacji.

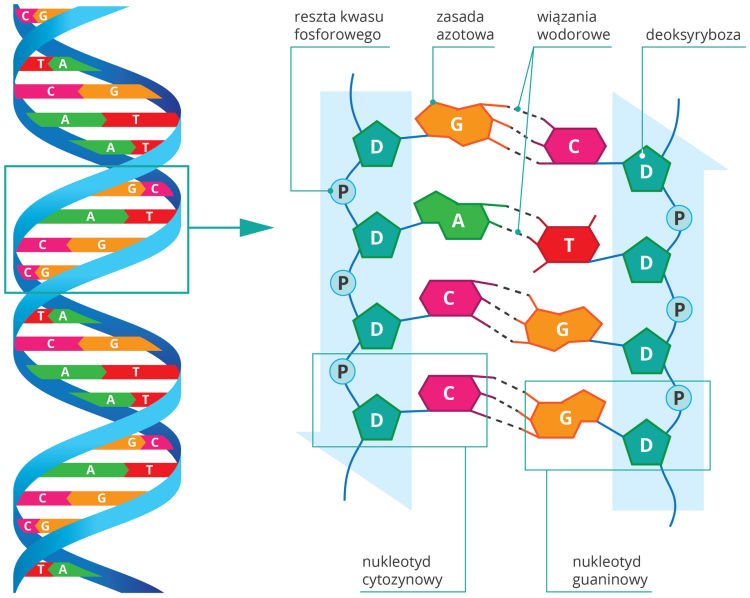
DNA wyróżnia się niezwykle wysoką gęstością zapisu danych oraz w odpowiednich warunkach dużą stabilnością przechowywania informacji w czasie [1]. Te cechy sprawiają, że staje się on interesującym kandydatem do pełnienia funkcji alternatywnego nośnika danych – zwłaszcza w kontekście rozwiązań, w których kluczowe znaczenie mają bezpieczeństwo, trwałość i kompaktowość reprezentacji danych.

Celem niniejszego rozdziału jest wprowadzenie do zagadnień związanych z właściwościami DNA, jego potencjałem jako nośnika informacji oraz sposobami kodowania danych cyfrowych w strukturze biologicznej. W rozdziale omówione zostaną podstawowe pojęcia związane z DNA i jego wykorzystaniem w kontekście zapisu danych.

5:2912668143

Budowa DNA

Cząsteczki DNA (ang. deoxyribonucleic acid), czyli cząsteczki kwasu deoksyrybonukleinowego, znajdują się w każdej żywej komórce. Zawierają one informacje o budowie i funkcjonowaniu organizmu [2].



Rysunek 1.1: Ilustracja fragmentu nici DNA [2].

Rysunek 1.1 przedstawia fragment cząsteczki DNA, która ma strukturę podwójnej helisy. Składa się ona z dwóch nici ułożonych antyrównolegle, połączonych ze sobą wiązaniami wodorowymi tworzącymi pary komplementarnych zasad azotowych.

Podstawową jednostką strukturalną DNA jest **nukleotyd**, który zbudowany jest z trzech elementów:

- jednej z czterech zasad azotowych:
  - **A** – adeniny,
  - **C** – cytozyny,
  - **G** – guaniny,
  - **T** – tyminy,
- cukru – deoksyrybozy,
- reszty kwasu fosforowego.

Zasady azotowe łączą się ze sobą w sposób specyficzny, tworząc pary komplementarne: adenina (A) zawsze łączy się z tyminą (T) za pomocą dwóch wiązań wodorowych, natomiast cytozyna (C) z guaniną (G) poprzez trzy wiązania wodorowe.

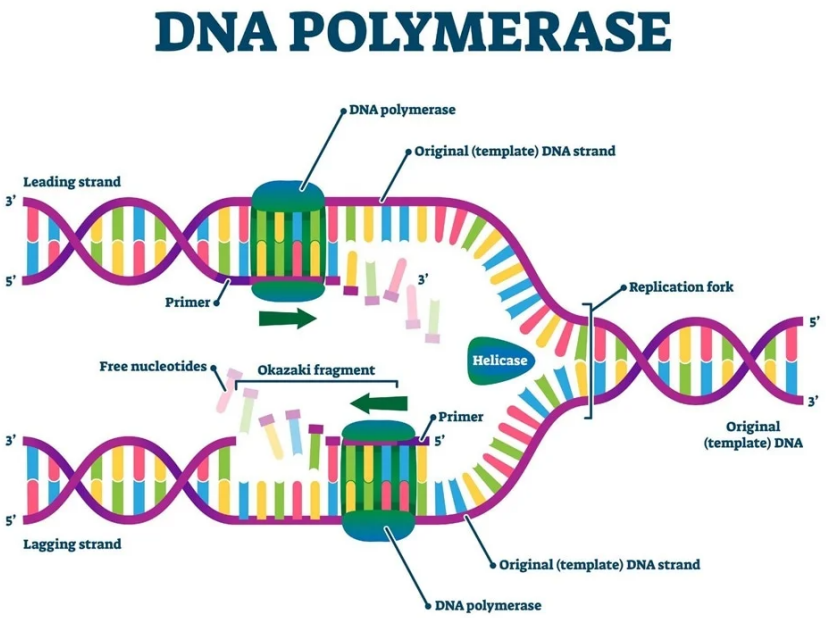
6:3820616332

Taki układ gwarantuje stabilność cząsteczki DNA oraz dokładność procesu jej replikacji. Warto podkreślić, że nici DNA są względem siebie komplementarne, a nie identyczne, co wynika bezpośrednio z reguł parowania zasad azotowych i przeciwnej orientacji łańcuchów.

Replikacja DNA

Replikacja DNA to proces endoenergetyczny, który wymaga dostarczenia energii z otoczenia. Jest to kluczowy etap w przekazywaniu informacji genetycznej, ponieważ przed każdym podziałem komórki — zarówno mitozą, jak i mejozą — konieczne jest wytworzenie identycznej kopii materiału genetycznego. Dzięki temu każda nowo powstała komórka otrzymuje pełny zestaw informacji zawartych w DNA komórki macierzystej.

Proces replikacji zachodzi w interfazie cyklu komórkowego, czyli jeszcze przed rozpoczęciem właściwego podziału. W komórkach eukariotycznych w których uczestniczą liczne **enzymy,** m.in. *polimeraza DNA, helikaza, primaza i ligaza,* odpowiadające za rozplatanie nici, syntezę nowych fragmentów DNA oraz ich łączenie w całość.



Rysunek 1.2: Replikacja DNA [3].

Replikacja DNA rozpoczyna się w specyficznym miejscu zwanym początkiem replikacji (ang. replication origin), gdzie enzymy inicjujące rozpoznają odpowiednią sekwencję DNA. Proces ten w schematyczny sposób został przedstawiony na rysunku 1.2.

- W pierwszym etapie replikacji niezbędny jest enzym o nazwie **helikaza,** który rozplata podwójną helisę, rozdzielając obie nici i tworząc tzw. widełki replikacyjne

7:1159559917

(ang. replication fork). W wyniku rozdzielenia powstają dwie pojedyncze nici DNA: jedna z nich jest odczytywana w kierunku określanym jako 5' → 3', a druga w przeciwnym 3' → 5'. Oznacznie te odnoszą się do kierunku przebiegu syntezy nowej nici, wynikającego z budowy chemicznej nukleotydów - różnicy między atomami węgla 5' i 3' w cząsteczce deoksyrybozy - i mają kluczowe znaczenie dla działania enzymów replikacyjnych, które mogą dołączać nowe nukleotydy wyłącznie do wolnej grupy hydroksylowej przy węglu 3'. Aby zapobiec ponownemu łączeniu się rozdzielonych nici, specjalne białka (ang. SSB proteins) stabilizują je, a topoizomeraza redukuje naprężenia wynikające z rozwijania helisy.

- W kolejnym kroku prymaza syntetyzuje krótkie odcinki RNA, zwane **primerami** (ang. primer), które są widoczne na schemacie jako krótkie fragmenty dołączone do nici DNA. Są one niezbędne, ponieważ **polimeraza DNA** (ang. DNA polymerase) wymaga startera, aby rozpocząć syntezę nowej nici DNA.

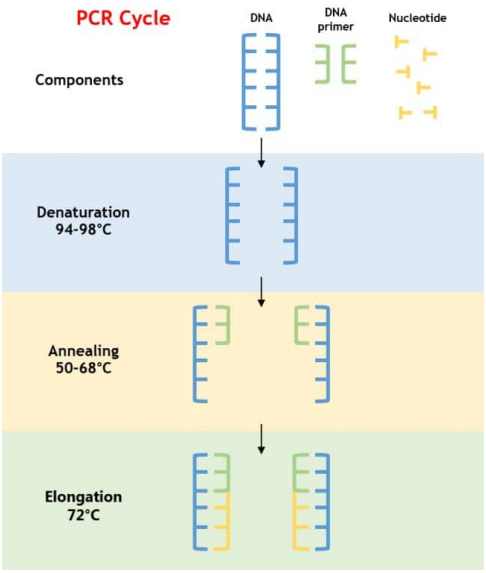
- Na nici wiodącej (ang. leading strand), która ma orientację 3' → 5', polimeraza DNA syntetyzuje nową nić DNA w sposób ciągły w kierunku 5' → 3'. Inaczej wygląda sytuacja na nici opóźnionej (ang. lagging strand), gdzie synteza zachodzi w przeciwnym kierunku do rozwijania widełek replikacyjnych. W efekcie polimeraza DNA tworzy krótkie fragmenty zwane fragmentami Okazaki (ang. Okazaki fragments), widoczne na rysunku jako oddzielne odcinki DNA.

- Każdy z tych fragmentów wymaga nowego startera RNA, co wymusza ciągłe przesuwanie primazy i polimerazy DNA. Po zakończeniu syntezy **ligaza** (ligase) usuwa luki między fragmentami Okazaki i łączy je w jedną, ciągłą nić DNA.

W efekcie końcowym powstają dwie identyczne cząsteczki DNA, z których każda składa się z jednej nici macierzystej oraz jednej nici nowo zsyntetyzowanej, będącej jej kopią. Dzięki temu informacja genetyczna zostaje powielona i wiernie przekazana do komórek potomnych.

8:6114165062

Reakcja łańcuchowa polimerazy (PCR)



Rysunek 1.3: Cykl procedury PCR [4].

Reakcja łańcuchowa polimerazy (ang. Polymerase Chain Reaction) – PCR jest techniką szeroko stosowaną w bioinżynierii i inżynierii molekularnej. PCR pozwala na szybką i tanią amplifikację fragmentów DNA, ponieważ jest w stanie wygenerować duże ilości określonych kopii fragmentów DNA z początkowo małej ilości bazowego DNA.

PCR jest serią powtarzających się cykli, odbywa się od 20 do 40 cykli replikacji DNA przez enzym polimerazy DNA. Po każdym cyklu, liczba nici DNA jest w przybliżeniu podwojona, więc przyjmując, że takich cykli będzie 40, jesteśmy w stanie wygenerować 1 bilion kopii z jednej nici DNA, gdyby wszystko przebiegło idealnie. Ilość materiału, w przybliżeniu wzrasta wykładniczo.

Każdy cykl PCR jest podzielony na cztery etapy:

- **Inicjalizacja** – próbka jest podgrzewana do 94–96°C przez 2–10 minut w celu aktywacji polimerazy DNA i zapewnienia odpowiednich warunków reakcji.
- **Denaturacja** – w tym etapie próbka jest podgrzewana do 94–98°C przez 20–30 sekund, co powoduje zerwanie wiązań wodorowych między komplementarnymi nićmi DNA. W efekcie dwuniciowa cząsteczka DNA rozdziela się na pojedyncze nici, które posłużą jako matryca w kolejnych etapach.
- **Hybrydyzacja** – jest to etap przyłączenia primerów, krótkich odcinków jednoniciowego DNA (zwykle o długości 18–25 nukleotydów), które są zaprojektowane tak, aby komplementarnie wiązać się z określonymi sekwencjami na matrycowej nici DNA.

9:4967225605

Po denaturacji primery pełnią kluczową rolę w reakcji PCR – stanowią punkt startowy dla polimerazy DNA, która nie może rozpocząć syntezy nowej nici bez wcześniej przyłączonego startera. Ich sekwencje są specyficzne dla amplifikowanego fragmentu DNA, co zapewnia precyzyjność reakcji.

W tym kroku temperatura jest obniżana do 50–68°C na 20–40 sekund, co umożliwia optymalne wiązanie primerów z matrycowym DNA i przygotowuje nici do syntezy.

- **Elongacja** – etap syntezy nowej nici i amplifikacji docelowego fragmentu. Polimeraza DNA dobudowuje nową nić w kierunku 5' → 3', wykorzystując trifosforany deoksynukleotydów (dNTP). Proces ten zachodzi w temperaturze około 72°C, a jego tempo zależy od długości amplifikowanego fragmentu i wydajności enzymu.

1.2 Zastosowania syntetycznego DNA

Jednym z innowacyjnych rozwiązań jest możliwość stworzenia syntetycznego DNA, które może być wykorzystywane w różnych dziedzinach, takich jak przechowywanie danych, znakowanie produktów jak i przetwarzanie sygnałów. DNA charakteryzuje się bardzo wysoką gęstością informacji, a także niskim zużyciem energii podczas długoterminowego przechowywania [5]. Dodatkowo jest biodegradowalne, co czyni je atrakcyjnym nośnikiem informacji z perspektywy zrównoważonego rozwoju.

Nośnik ten umożliwia zapis ogromnych ilości informacji przy bardzo wysokiej gęstości danych, a w odpowiednich warunkach może również zapewnić ich długoterminowe przechowywanie. Warto jednak zaznaczyć, że DNA jest materiałem podatnym na degradację biologiczną i chemiczną, dlatego jego trwałość zależy w dużej mierze od sposobu zabezpieczenia i środowiska przechowywania (np. sucha, chłodna i beztlenowa atmosfera) [6].

Pomimo że technologia DNA ma wiele zalet, to jednak posiada ona również liczne wady, które wymagają usprawnienia i są nimi m.in. różnica w kosztach i szybkości zapisu oraz odczytu danych pomiędzy DNA a technologiami konwencjonalnymi. Jednak nowe techniki syntezy DNA, takie jak synteza enzymatyczna, mogą przyczynić się do obniżenia kosztów i zwiększenia szybkości zapisu oraz odczytu danych [7].

Zastosowanie DNA w sektorze IT

Sektor IT (technologii informacyjnych) rozwija się dynamicznie, a technologie takie jak przechowywanie danych w DNA czy znaczniki DNA, choć nadal we wczesnej fazie rozwoju, mogą znaleźć zastosowanie w przyszłości jako uzupełnienie istniejących rozwiązań informatycznych. Obszary z pogranicza biologi i informatyki, takie jak bioinformatyka czy biokryptografia – zwłaszcza w zakresie analizy danych, algorytmów kodowania czy integracji systemów – odgrywają istotną rolę w rozwijaniu i wdrażaniu tych technologii.

10:6553831560

- **Cyberbezpieczeństwo i autoryzacja danych** – w tej sferze DNA może zostać wykorzystywane do generowania unikalnych identyfikatorów i kluczy kryptograficznych, co może znacznie wpłynąć na poprawę bezpieczeństwa systemów informatycznych. Wprowadzenie indywidualnych znaczników DNA w chipach zabezpieczających lub nośnikach danych może utrudnić ataki hackerskie i fałszerstwa.

- **Zarządzanie tożsamością i dostępem** – Technologia DNA może zostać wykorzystana jako nowa forma biometrycznej autoryzacji użytkowników. W przyszłości możliwe jest stworzenie systemów uwierzytelniania, które zamiast tradycyjnych haseł lub skanów odcisków palców, będą wykorzystywać fragmenty unikalnego DNA użytkownika do potwierdzania tożsamości.

- **Przechowywanie danych w DNA** – DNA oferuje niezwykle wysoką gęstość zapisu informacji. Przechowywanie danych w postaci sekwencji nukleotydów stanowi potencjalne rozwiązanie dla rosnących problemów związanych z ograniczoną pojemnością tradycyjnych nośników pamięci, a także ich degradacją w czasie [8].

- **Internet Rzeczy (IoT) i inteligentne systemy śledzenia** – Technologia DNA może zostać zintegrowana z systemami IT stosowanymi w IoT, umożliwiając unikalne znakowanie produktów oraz autoryzację transakcji między urządzeniami. Znaczniki DNA mogą być nanoszone na komponenty elektroniczne, sprzęt przemysłowy oraz elementy infrastruktury krytycznej, umożliwiając ich identyfikację w globalnych systemach zarządzania zasobami [9].

- **Analiza Big Data i bioinformatyka** – Wykorzystanie DNA w IT nie ogranicza się jedynie do znakowania czy kryptografii. Zastosowanie metod bioinformatycznych w analizie ogromnych zbiorów danych może znaleźć zastosowanie w sektorze zdrowotnym, genetyce i badaniach nad personalizowaną medycyną.

Rozwój tych technologii wymaga dalszych badań i testów, jednakże DNA jako nośnik informacji oraz narzędzie zabezpieczające ma potencjał, aby stać się kluczowym elementem przyszłości IT.

Zastosowania DNA poza sektorem IT

DNA w postaci specjalnych znaczników może znaleźć zastosowanie nie tylko w sektorze IT, lecz także w innych branżach. Przykładem jest przemysł tekstylny, gdzie w Indiach i innych krajach stosuje się znaczniki DNA do znakowania surowej bawełny. Umożliwia to identyfikację źródła jej pochodzenia, co pozwala producentom odzieży zagwarantować klientom, że oferowana bawełna organiczna rzeczywiście pochodzi ze sprawdzonych upraw i nie została rozcieńczona syntetycznymi włóknami [10].

11:1787742247

Oprócz zastosowań przemysłowych, znaczniki DNA znalazły również bardziej nietypowe zastosowania. Przykładem jest historia Joe Barbery, współtwórcy kultowych animacji takich jak *Flintstonowie, Jetsonowie* i *Scooby-Doo*. W 1995 roku Barbera zwrócił się do firmy DNA Art Guard International z prośbą o wykorzystanie jego DNA do stworzenia unikalnego atramentu. Firma, wykorzystując technikę PCR, zmieszała jego sekwencję DNA ze specjalnym tuszem, umożliwiając zapisanie kodu genetycznego w formie pisemnej. Charles Butland, będący twórca tego rozwiązania, podkreślał, że „technika nie może być powielona, chyba że kopista zna sekwencję oraz startery" [11].

Tagi DNA jako metoda znakowania i weryfikacji autentyczności

DNA może być wykorzystywane jako unikalny znacznik, umożliwiający śledzenie produktów oraz potwierdzanie ich autentyczności. Technologia ta znajduje zastosowanie w wielu branżach, zapewniając skuteczne zabezpieczenia przed fałszerstwami i pozwalając na identyfikację pochodzenia surowców.

Na rynku dostępne są już komercyjne rozwiązania wykorzystujące DNA jako narzędzie znakowania. Przykładem może być firma Applied DNA Sciences, która opracowała metodę syntetycznego DNA do znakowania produktów przemysłowych, tekstyliów czy komponentów elektronicznych. Dzięki unikalnym sekwencjom możliwe jest jednoznaczne potwierdzenie autentyczności oznakowanych dóbr [12].

Innym przykładem jest SelectaDNA, technologia stosowana m.in. w systemach zabezpieczeń pojazdów. Preparat zawierający mikrokropki z unikalnym kodem DNA nanoszony jest na elementy samochodu, co umożliwia ich identyfikację w przypadku kradzieży. Tego typu rozwiązania są już wykorzystywane przez służby porządkowe i firmy ubezpieczeniowe jako skuteczna metoda prewencji przestępczości [13].

1.3 Zapis informacji w DNA

Współczesna technologia umożliwia wykorzystanie syntetycznego DNA jako nośnika informacji, niezależnie od jego biologicznej funkcji. Oznacza to, że cząsteczka, która przez miliony lat pełniła funkcję nośnika informacji genetycznej, może zostać zastosowana jako alternatywny system zapisu danych, który w przyszłości może pełnić funkcję analogiczną do dzisiejszych nośników pamięci masowej. Powstało wiele prac, w których autorzy proponują różne rozwiązania pozwalające wykorzystać DNA jako nośnik do zapisu informacji, np. [6], [14]. Niektóre rozwiązania pozwalają nawet na realizację zapisu i odczytu [15].

Istotnym elementem procesu umożliwiającego zapis danych w DNA jest synteza sztucznego DNA, które można wykorzystać do przechowywania dowolnych informacji.

Proces zapisu informacji w nici DNA składa się z kilku etapów. Jednym z kluczowych elementów tego procesu jest konwersja danych do postaci binarnej, czyli sekwencji zer

12:1867152317

i jedynek, ponieważ umożliwia ona dalsze przekształcenie danych cyfrowych w sekwencje nukleotydów, które mogą być syntetyzowane jako cząsteczki DNA. Następnie dane binarne są przekształcane w sekwencję nukleotydów z wykorzystaniem czterech zasad azotowych: adeniny (A), cytozyny (C), guaniny (G) i tyminy (T). Proces ten umożliwia zakodowanie informacji cyfrowej w strukturze DNA i stanowi pierwszy etap konwersji danych.

Wykorzystanie DNA jako nośnika danych wymaga zagwarantowania jego stabilności w odpowiednich warunkach. Badania wskazują, że przy optymalnych parametrach przechowywania DNA może zachować swoją strukturę nawet przez tysiące lat, co potencjalnie może czynić je trwalszym nośnikiem informacji niż współczesne technologie, takie jak dyski twarde, płyty optyczne czy taśmy magnetyczne [8].

Ze względu na tę właściwość, DNA stanowi obiecującą alternatywę dla długoterminowej archiwizacji danych, szczególnie w kontekście przechowywania informacji historycznych, naukowych oraz związanych z dziedzictwem kulturowym.

Cykl przetwarzania, przechowywania i odczytu informacji w DNA obejmuje następujące etapy:

- **Konwersja danych cyfrowych na sekwencje nukleotydów** – dane cyfrowe są najpierw przekształcane w sekwencje binarne (ciągi zer i jedynek). Następnie kod binarny jest konwertowany na sekwencje nukleotydowe (A, T, C, G).

  Przykładowo, w podstawowym schemacie kodowania można przypisać poszczególnym parom bitów określone nukleotydy:

  - `00` → A,
  - `01` → T,
  - `10` → C,
  - `11` → G.

  Tego rodzaju podejście jest jednak bardzo elementarne i nie uwzględnia ograniczeń biologicznych ani ryzyka błędów podczas syntezy i odczytu DNA. Zaawansowane systemy kodowania danych w DNA, takie jak DNA Fountain, wykorzystują mechanizmy redundancji, sum kontrolnych oraz kodowanie korekcyjne (np. kody Luby'ego – LT codes), co pozwala zwiększyć niezawodność transmisji informacji i odporność na uszkodzenia sekwencji [6].

- **Synteza DNA** – to proces „sztucznej produkcji" sekwencji DNA metodami chemicznymi lub enzymatycznymi. W kontekście zapisu danych polega on na przekształceniu zakodowanej informacji cyfrowej (np. ciągu nukleotydów odpowiadającego konkretnym wartościom binarnym) w rzeczywistą cząsteczkę DNA. Na początku syntetyzowane są krótkie odcinki zwane oligonukleotydami, które następnie

13:4874843634

łączy się w dłuższe fragmenty za pomocą enzymów, np. ligazy DNA. Technologia ta umożliwia fizyczną realizację procesu zapisu danych w materiale biologicznym, co czyni ją kluczowym etapem w tworzeniu DNA jako alternatywnego nośnika informacji.

- **Przechowywanie DNA** – zsyntetyzowane DNA przechowuje się w specjalnych warunkach zapewniających jego długoterminową stabilność. Najczęściej stosuje się formę suchą w niskiej temperaturze lub przechowywanie w mikrofiolkach. Alternatywne metody obejmują zamknięcie DNA w silikonowych kapsułkach lub nanostrukturach, zapewniających ochronę przed degradacją oraz umożliwiających długoterminową archiwizację danych [8].

- **Odczytywanie z DNA** – aby odzyskać zapisane informacje, DNA poddaje się procesowi sekwencjonowania, czyli w najprostrzym ujęciu, procesowi odczytywania kolejności zasad azotowych w nici DNA. Uzyskana sekwencja nukleotydów jest następnie przekształcana z powrotem na format binarny, zgodnie z wcześniej przyjętym schematem kodowania. Odczyt ten stanowi końcowy etap cyklu, umożliwiając uzyskanie danych cyfrowych z fizycznego nośnika opartego na DNA.

1.4 Kryptografia w DNA

Kryptografia to dziedzina nauki zajmująca się metodami zabezpieczania informacji między innymi poprzez ich szyfrowanie. Polega na przekształceniu danych w taki sposób, aby były one dostępne jedynie dla uprawnionych odbiorców posiadających odpowiedni klucz deszyfrujący.

Podstawowym celem kryptografii jest zapewnienie poufności, integralności, autentyczności i niezaprzeczalności danych. Mechanizmy kryptograficzne znajdują zastosowanie w różnych obszarach, m.in. w komunikacji, ochronie danych osobowych, transakcjach internetowych oraz przechowywaniu informacji w sposób zabezpieczony przed nieautoryzowanym dostępem.

1.4.1 Tradycyjne metody kryptografii

Siła kryptografii zależy od mocy obliczeniowej komputerów oraz złożoności klucza szyfrującego. Długość klucza, wyrażona w bitach, ma kluczowe znaczenie dla bezpieczeństwa – im dłuższy klucz, tym większa liczba możliwych kombinacji, co utrudnia jego złamanie metodami ataku siłowego (bruteforce). Dla osoby nieuprawnionej złamanie długiego klucza kryptograficznego jest procesem czasochłonnym i wymaga dużej mocy obliczeniowej. Istotnym wyzwaniem jest również bezpieczne przechowywanie kluczy. Ze względu na ich złożoność i długość, użytkownicy nie są w stanie ich zapamiętać, dlatego konieczne

14:6042736513

jest ich zapisanie. Klucze te są najczęściej przechowywane w dedykowanych systemach zarządzania kluczami, które same podlegają dodatkowemu zabezpieczeniu, np. krótszym hasłem możliwym do zapamiętania przez użytkownika.

Przechowywanie kluczy kryptograficznych na dyskach twardych lub nośnikach zewnętrznych, takich jak pendrive'y, niesie jednak ryzyko zarówno fizyczne (np. kradzież, uszkodzenie nośnika), jak i cybernetyczne (np. ataki elektromagnetyczne lub złośliwe oprogramowanie). Jeśli klucz jest przechowywany na urządzeniu podłączonym do Internetu, dochodzą dodatkowe zagrożenia, takie jak podatności sieciowe, ataki na oprogramowanie czy sprzęt.

1.4.2 Kryptografia oparta na DNA

Ze względu na specyficzne właściwości biologicznego nośnika, jakim jest DNA, coraz częściej rozważa się jego zastosowanie w kontekście przechowywania kluczy kryptograficznych. W przeciwieństwie do tradycyjnych nośników półprzewodnikowych, DNA charakteryzuje się wysoką odpornością na zakłócenia elektromagnetyczne, możliwością ukrycia w mikroskali oraz trudnym dostępem fizycznym, co czyni go atrakcyjnym z punktu widzenia bezpieczeństwa informacji.

Jedną z możliwych metod przechowywania kluczy kryptograficznych jest ich zapis w strukturze DNA. Podejście to oferuje wysoki poziom bezpieczeństwa, minimalizując ryzyko przechwycenia klucza przez osoby nieuprawnione. DNA jest również mniej podatne na zakłócenia elektromagnetyczne niż tradycyjne nośniki cyfrowe, co zwiększa jego odporność na niektóre typy ataków fizycznych.

Ponadto, dzięki unikalnym właściwościom DNA, informacja kryptograficzna może być replikowana w dużej liczbie kopii, jednocześnie zajmując minimalną przestrzeń fizyczną na poziomie molekularnym. W kwestii bezpieczeństwa ważne może być wyeliminowanie stron trzecich, pośredniczących w procesie tworzenia klucza, przenosząc istotne etapy procesu jego tworzenia tylko na właściciela [15].

Klucze kryptograficzne mogą być przechowywane jako ciągi bitów zakodowanych w sekwencjach nukleotydowych. Proces ten wymaga jednak odpowiedniego doboru sekwencji nukleotydów w nici DNA, aby zapewnić trwałość i integralność zapisanej informacji oraz możliwość jej precyzyjnego odczytu w przyszłości.

15:2340224562

Rozdział 2

Aplikacja do zarządzania danymi zapisanych w DNA

Zapis danych w strukturze nici DNA, a w szczególności kluczy kryptograficznych, jest procesem wieloetapowym i złożonym. Obejmuje on m.in. konwersję danych binarnych do sekwencji nukleotydów, ich syntezę, przechowywanie, a także późniejsze odczytywanie i dekodowanie. Każdy z tych etapów wymaga precyzyjnego zarządzania informacją, kontroli nad integralnością danych oraz możliwość wizualizacji i weryfikacji poszczególnych kroków.

Z powyższych względów powstał pomysł opracowania dedykowanego narzędzia wspomagającego zarządzanie całym cyklem życia danych zapisywanych w DNA – od ich wygenerowania po integrację z innymi systemami. Przykładem realizacji tego pomysłu jest aplikacja webowa, która udostępnia użytkownikowi intuicyjny interfejs pozwalający na konwertowanie danych (np. kluczy krypotgraficznych) do postaci sekwencji DNA oraz zarządzanie nimi.

Opis działania aplikacji oraz szczegółowy zakres jej funkcjonalności przedstawiono w kolejnych podrozdziałach.

16:4220931163

2.1 Opis problemu

Wraz z dynamicznym rozwojem technologii kodowania informacji w strukturze syntetycznego DNA, rośnie również potrzeba tworzenia narzędzi, które umożliwią efektywne zarządzanie danymi przechowywanymi w tej formie. Jednym z przykładów zastosowania takiego kodowania może być przechowywanie kluczy kryptograficznych w formie sekwencji nukleotydowych. Chociaż proces zapisu danych w DNA został szczegółowo opisany w literaturze, pozostaje on technicznie złożony, wieloetapowy a tym samym trudny do bezpośredniego wykorzystania przez użytkowników nieposiadających specjalistycznej wiedzy technicznej.

Cały proces obejmuje między innymi:

- konwersję danych binarnych do sekwencji nukleotydów (ciągów złożonych z liter A, C, T i G),
- strukturalne uformowanie nici DNA (w tym, jeśli zachodzi taka potrzeba, przypisywanie tzw. primerów, wypełnień oraz innych informacji związanych z wykorzystaną metodą kodowania danych),
- przechowywanie oraz wizualizację uzyskanych danych w sposób umożliwiający ich dalsze wykorzystanie np. w procesach znakowania biologicznego lub identyfikacji.

Poza kilkoma zamkniętymi rozwiązaniami komercyjnymi, takimi jak HelixWorks[1], Applied DNA Sciences czy SigNature DNA[2], pozostałe dostępne rozwiązania mają charakter naukowy i funkcjonują najczęściej jako systemy typu proof of concept. Zazwyczaj są to zestawy skryptów wykorzystywane w środowiskach badawczych, których obsługa wymaga zaawansowanej wiedzy technicznej. Brakuje natomiast intuicyjnych i zorientowanych na użytkownika narzędzi konsumenckich, które pozwalałyby na łatwe generowanie, organizowanie i przeglądanie danych DNA.

W odpowiedzi na te potrzeby, niniejsza praca podejmuje próbę stworzenia aplikacji webowej pełniącej funkcję interfejsu użytkownika, będącego częścią szerszego systemu umożliwiającego przystępne korzystanie z istniejącego już mechanizmu kodowania danych w sekwencji DNA.

Projektowana aplikacja:

- nie zajmuje się bezpośrednio procesem konwersji danych – ten został zaimplementowany wcześniej – lecz integruje go w ramach graficznego interfejsu,
- umożliwia generowanie sekwencji nukleotydowych o określonej strukturze na podstawie danych wejściowych,

[1] https://www.helix.works/

[2] https://adnas.com/apdn-signs-crada-with-us-dept-of-agriculture/

17:5061113416

- przypisuje je do kont użytkowników,
- umożliwia przeglądanie i organizowanie danych w czytelny sposób,
- oferuje eksport nici do dalszego wykorzystania.

2.2 Mechanizm kodowania danych przez użytkownika w DNA

Jednym z najważniejszych elementów całęgo procesu zarządzania jest konwersja danych cyfrowych do sekwencji nukleotydów tworzących nić DNA. Mechanizm ten został zaimplementowany w języku Python w oparciu o opis znajdujący się w [15].

Kodowanie danych wejściowych - skrypt `bits.py`

Proces zapisywania danych w DNA przebiega w następujących krokach:

1. **Konwersja danych**
   Wprowadzone przez użytkownika dane, np. tekst konwertowany jest do postaci binarnej, przy użyciu wbudowanych funkcji Pythona.

2. **Podział danych binarnych**
   W procesie kodowania dane binarne dzielone są na bloki o długości **k** bitów, przy czym **k** definiuje, ile informacji można zakodować w jednej sekwencji DNA. Na tej podstawie określana jest liczba wymaganych, różnych sekwencji. W przypadku, gdy ilość bitów danych nie jest podzielna przez **k**, ostatnia sekwencja zostaje uzupełniona losowymi nukleotydami w celu zachowania odpowiedniej długości.

3. **Budowa struktury nici**
   Każda sekwencja budowana jest według schematu:

   $$S - B_1 - s_1 - B_2 - s_2 - \ldots - s_{k-1} - B_k - E$$

   Gdzie:

   - S, E – primer początkowy i końcowy (losowo dobrane z dostępnego zbioru) [16],
   - $B_1, B_2, \ldots, B_k$ – sekwencja primerów reprezentująca bit (0 lub 1),
   - s_1, s_2 – losowa sekwencja o długości 20 nukleotydów separująca kolejne bity.

4. **Wielokrotne nici**
   Dla danych nie dających się zakodować w pojedyńczej nici tworzona jest odpowiednia liczba nici. Struktura dla nici m-tej wygląda następująco:

   $$S_m - B_{1:m} - s_{1:m} - B_{2:m} - s_{2:m} - \ldots - s_{k-1:m} - B_{k:m} - E_m$$

18:9985198654

Wybrana struktura danych i sposób ich kodowania są ściśle związane ze sposobem odczytu, co jest podyktowane kwestiami bezpieczeństwa. W przytaczanym przypadku jest to PCR oraz analiza rozkładu prążków na żelu powstałych w procesie elektroforezy. Dane wejściowe w strukturze DNA kodowane są z wykorzystaniem skryptu napisanego w pythonie `bits.py`.

Przykładowy wynik działania skryptu przedstawiono na rysunku 2.1 – pokazuje on dokładną strukturę nici w których zakodowano dane użytkownika:

- `Bits: 35` – liczba bitów do zakodowania (w tym przykładzie to 35),
- `Sub len: 8` – liczba bitów kodowana w pojedyńczej nici DNA,
- `Number of sub: 5` – całkowita liczba nici koniecznych do zakodowania danych użytkownika,
- `Sub: [...]` – sekwencja (w podziale na odcinki o długości 20nt) reprezentująca nić DNA wraz z zakodowanymi informacjami.

```
Bits: 35
Sub len: 8
Number of sub: 5
5
Sub: ['CATAAAATTGTCCGCGCCAC', 'GTATAGCCTTCGAATACGGC', 'CTCTGTCGTCTCTGTTGGAA', 'ATGGTGTACGCAAACGGAAG', 'TACTCGCGCAAGTGAGA
TCA', 'CGCTCGTTCTCATTGCTATC', 'TACTCGCGCAAGTGAGATCA', 'GGGCCGATGTTGTTTATCTG', 'CTCTGTCGTCTCTGTTGGAA', 'CCAAGATGTCGATCATA
CGC', 'CTCTGTCGTCTCTGTTGGAA', 'TACCGCGGTTTAATCACACG', 'TACTCGCGCAAGTGAGATCA', 'AAGCTAGACACAGGCCTTTC', 'CTCTGTCGTCTCTGTTG
GAA', 'CTAGTAAGGGACGGCAAATC', 'TACTCGCGCAAGTGAGATCA', 'GAGATAATAGTGGCGCGCTT', 'ACTATATTGACGAGGAGCCG']
Sub: ['CCACGGATTAACAGGCAAGA', 'GAGATAATAGTGGCGCGCTT', 'CTCTGTCGTCTCTGTTGGAA', 'ACGAATCATGACCTGGCTAC', 'CTCTGTCGTCTCTGTTG
GAA', 'CGGGAGAAAACACATGGCTA', 'TACTCGCGCAAGTGAGATCA', 'CTCTACGAGGCTCATAGGAA', 'CTCTGTCGTCTCTGTTGGAA', 'AAGAGGTAGACCCCTCC
TAT', 'TACTCGCGCAAGTGAGATCA', 'TCACGGTAGTAGGGTAACGA', 'CTCTGTCGTCTCTGTTGGAA', 'TACGGGAGTGTTTCTTGGAC', 'TACTCGCGCAAGTGAGA
TCA', 'TCAGTTTCCACTTCGTCTCC', 'CTCTGTCGTCTCTGTTGGAA', 'TTGTGGAGTCCGAACACAAG', 'CTCCATTCGCTTACTGTCCT']
Sub: ['GGATCTTGCGTTTGCTAGAG', 'TTGTGGAGTCCGAACACAAG', 'TACTCGCGCAAGTGAGATCA', 'GACCTATCATTACCGCATGC', 'TACTCGCGCAAGTGAGA
TCA', 'CGTAGCACCTCTATCCGATA', 'CTCTGTCGTCTCTGTTGGAA', 'CTGAGAGAGTAGTACGCCAA', 'TACTCGCGCAAGTGAGATCA', 'AACCATCGTCGAACATC
CTC', 'CTCTGTCGTCTCTGTTGGAA', 'GTATCGCGCATTACCCAAAG', 'TACTCGCGCAAGTGAGATCA', 'AGAATGCTCTCAGCGGTAGA', 'CTCTGTCGTCTCTGTTG
GAA', 'TCGACCTATTGTACTTGCCC', 'TACTCGCGCAAGTGAGATCA', 'CTCGCGTTAGGGACGTAAAT', 'TCGGAGAATTACCCTCTGCA']
Sub: ['CGTTTTGAAGCCGGAAGATC', 'CTCGCGTTAGGGACGTAAAT', 'CTCTGTCGTCTCTGTTGGAA', 'ACGCTTAGACTCTACACGTG', 'TACTCGCGCAAGTGAGA
TCA', 'CTCGTTGAGAGATGGATCCA', 'CTCTGTCGTCTCTGTTGGAA', 'AAAGGTGGCGATCACTCAAC', 'TACTCGCGCAAGTGAGATCA', 'CAGTTCATCGACTGTTT
GCC', 'CTCTGTCGTCTCTGTTGGAA', 'CCTTGAGCGGTTTACGACAT', 'TACTCGCGCAAGTGAGATCA', 'TATATGGGATGGTTACCCCG', 'CTCTGTCGTCTCTGTTG
GAA', 'ATAGCGAACCCCAACCTTAC', 'TACTCGCGCAAGTGAGATCA', 'TTCTGTGACCTGCCGTAGAA', 'TGGATTAGCGATGCTTCCCT']
Sub: ['AGAGGTCCTCTTGTTTTCCC', 'TTCTGTGACCTGCCGTAGAA', 'CTCTGTCGTCTCTGTTGGAA', 'CCTAGGTTGGCGGTTATCTT', 'TACTCGCGCAAGTGAGA
TCA', 'AACCTTTTCGCGTCAAGGAG', 'CTCTGTCGTCTCTGTTGGAA', 'GTTAAGGCGTCAACATGCTG', 'CCACTCTGATATCCAGTCGT']
```

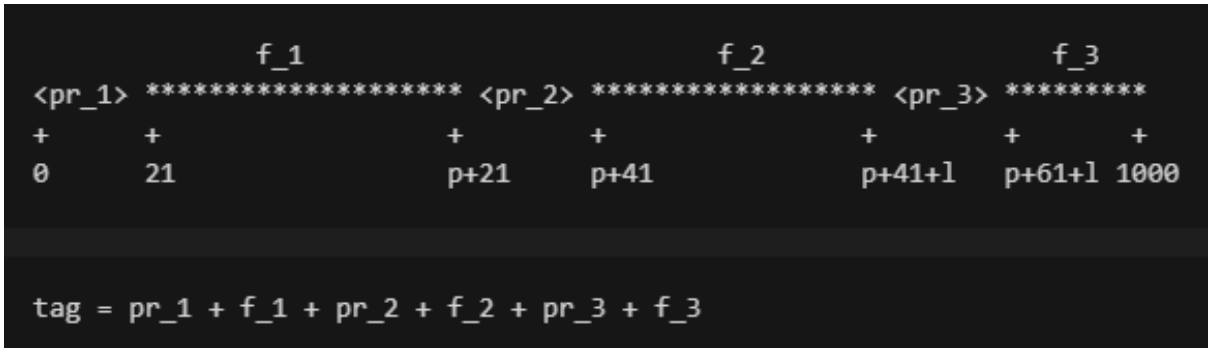
Rysunek 2.1: Rysunek przedstawia wynik działania skryptu `bits.py`, który dokonuje podziału i wizualizuje strukturę nici DNA.

Opisywana aplikacja webowa daje także możliwość tworzenia prostych znaczników DNA wykorzystywanych do oznaczania przedmiotów i ich weryfikacji [17].

Generowanie znacznika DNA

Tworzenie znacznika polega na wygenerowaniu fragmentu nici DNA (tzw. tagu), o długości 1000 nukleotydów, zawierającego losową sekwencję oraz specjalnie przygotowane miejsca wiązania primerów, umożliwiające późniejszą weryfikację metodą PCR. Schemat struktury takiego tagu przedstawiono na rysunku 2.2.

19:1070359235



Rysunek 2.2: Schemat nici DNA.

Wygenerowana sekwencja reprezentująca nić składa się z trzech obszarów wiązania primerów (ang. binding site) oraz trzech wypełnień utworzonych z losowych ciągów nukleotydów. Działanie skryptu można opisać w następujących krokach:

1. **Losowanie primerów**
   Z listy tysiąca dostępnych primerów [16] wybierane są losowo trzy sekwencje (`pr_1, pr_2, pr_3`), które będą rozmieszczone w różnych miejscach nici.

2. **Wyznaczenie długości fragmentów wypełniających**
   Losowane są dwie wartości:

   - `f_1` – liczba całkowita z przedziału 100–1000 (co 50),
   - `f_2` – liczba z przedziału `50 - (600 - f_1)`,
   - Trzeci fragment `f_3` jest wyliczany jako dopełnienie do długości 1000 nukleotydów: `f_3 = 1000 - (len(pr_1) + len(pr_2) + len(pr_3) + f_1 + f_2)`

3. **Generowanie sekwencji**
   Dla każdego z fragmentów `f_1, f_2, f_3` tworzony jest losowy ciąg z alfabetu nukleotydowego (A, C, T, G), a następnie wszystkie elementy są łączone w jedną nić: `pr_1 + f1_comp + pr_2 + f2_comp + pr_3 + f3_comp`

   `f_comp`, oznacza nie długość a sekwencję nukleotydów o długości `f`.

4. **Weryfikacja**
   Sekwencja może być sprawdzona poprzez reakcje PCR dla primerów `pr_1:pr_3` oraz `pr_2:pr_3`. Oczekiwany układ prążków na żelu powinien odpowiadać długościom `f_1` i `f_2`.

Do tego celu wykorzystano skrypt `simple_tag.py`.[3] Omawiany skrypt generuje sekwencję nukleotydową, która po syntezie w postaci nici DNA może być wykorzystana jako

[3]źródło pliku - `https://github.com/miskiewiczm/bits_concept/blob/main/simple_tag.py`

20:5435398744

prosty tag DNA. Taki tag lub znacznik, o ile zapewnione zostaną odpowiednie warunki biologiczne może zostać naniesiony na obiekty fizyczne.

Rysunek 2.3 przedstawia przykładowy wynik działania tego skryptu – czyli wygenerowaną strukturę taga DNA. W pokazanym przykładzie weryfikacja za pomocą PCR powinna wskazać aplikony o długościach około 300 i 600 nt.

```
CTGAGGAACCCAACTTGTAC - 300 - TACCCTAGGCAAACGTAGCA - 300 - CATTCCCGTCGGAGTTGAAA - 340
Verification codes:
1. 300
2. 600
3. 300
strand:
 CTGAGGAACCCAACTTGTACGCCGGCTCCCGGTGTGCTACGCGAATCGTATACGTACTAAAGCAAGACCGAAACGCGCTATGCTCCCCTTTGGTAAAGTG
AAGGCAGCATCACAGTGCCCAGGATCGAGCGAGTTTGATAGCGGTTCTTACCCTTTCAGTACTTCTGGGGCAGCTGGAACATCAAGTAGAGGCCTCATACA
GTCCCAGTTTGAGGCTATCCGTCTGCGTCAGACTTTTTTTCGTCTCTCCCCAGCTGTGACCAGGTTAACTGCTGGACAGCCTGCGACTCGACATACGTGCT
CCTCATCCGTTAGCGCGGTACCCTAGGCAAACGTAGCAAGTTGATGGGCGCTCCCGGCGAGTCCAGATTGACGTTTCTCTTAGTGAAAGCTGGCAAAACCG
GTGTGACCCGAATCCCGGGTCCACGACTCAAGCTCTAAAACGAGCAACTAAAAAGGTAGTTTGCCCCCTAGCTGCCACTTATTATCGCCATTCGACGACAC
TAGAATGCTAACTGTCTCGCCGCCTGTGGAGGTGCACGGGCCATCGGCTTTCAGCTATACTCCGCGCAAGTCCATTAATGCTGGGTACCAGCGACTATTTG
AAAATAAAAGTCTACTAACTTATCTTGCCTCTATTCATTCCCGTCGGAGTTGAAACCGTATCCGCTCACCGCGGACCTTCGACAGAGTCGTCGAGCGACTT
TGATGTTGGGTGCTCGTCCGCAGCTCACGAGCAGAGACCTTCGAGATCCAAGAGTGATGGTTTCCCTTACGCTCAGAATGCAACTCGACAACGCGAACGCT
CTTCGGTCGGGGACACCTGCTTGCAGGATCGCACGCCCTTCCTTGTCCTCCGCGCGTTATAACCTCCTGGTATCACGCGTATCTGCCTACCTTCATAAACT
GTTTTGATCCACACTACCTCGCGCCGACGATGTGACGCGACTTCGATCGCTGGATGGTTATTGTTAACTGGGTTACAGTTGACCTATAACCC
```

Rysunek 2.3: Wynik działania skryptu `simple_tag.py`, który wizualizuje sposób stworzenia tagu DNA.

Działanie obu skryptów jest wywoływane za pośrednictwem zapytań HTTP kierowanych do API aplikacji, dzięki czemu użytkownik nie musi znać szczegółów technicznych, aby wygenerować gotowe sekwencje DNA.

2.3 Opis aplikacji webowej

Projektowana aplikacja webowa ma na celu uproszczenie i uporządkowanie procesu zarządzania danymi kodowanymi w strukturze DNA, w tym na przykład kluczami kryptograficznymi. Zamiast ręcznego przetwarzania danych wyjściowych z istniejących skryptów, aplikacja oferuje zunifikowany interfejs, który umożliwia pełną obsługę cyklu życia danych – od momentu ich wygenerowania aż po przygotowanie do fizycznego oznaczenia DNA przez zewnętrzne firmy.
System został zaprojektowany w oparciu o trójwarstwową architekturę:

- warstwa danych – przechowuje informacje o użytkownikach oraz przypisane im sekwencje DNA,
- warstwa serwerowa (backend) – odpowiada za generowanie sekwencji oraz zarządzanie logiką biznesową,
- warstwa prezentacji (frontend) – umożliwia użytkownikowi przeglądanie, przypisywanie i pobieranie danych w sposób intuicyjny.

21:8042303793

Podstawowe funkcje systemu obejmują:

- tworzenie i zarządzanie kontami,
- generowanie sekwencji DNA na podstawie danych binarnych,
- przypisywanie sekwencji do kont użytkowników,
- przechowywanie i zarządzanie zarejestrowanymi sekwencjami,
- wizualizację danych w formie ułatwiającej interpretację i dalsze wykorzystanie,
- eksport danych w formacie umożliwiającym ich fizyczne odwzorowanie przez zewnętrzne urządzenia znakujące.

Dzięki temu aplikacja nie tylko ułatwia obsługę technicznych aspektów zapisu danych w DNA, ale również odpowiada na potrzeby użytkowników końcowych, którzy mogą zarządzać informacją w prosty i bezpieczny sposób, bez konieczności bezpośredniego kontaktu z kodem lub nieczytelnym wynikiem skryptu.

2.4 Użyte narzędzia

Opisywaną aplikację stworzono z wykorzystaniem różnych narzędzi programistycznych, w tym języków programowania, bibliotek oraz środowisk wspomagających tworzenie aplikacji webowych. Struktura systemu obejmuje trzy główne warstwy: warstwę prezentacji (frontend – React.js), warstwę logiki aplikacyjnej (backend – Django) oraz warstwę danych (baza danych – PostgreSQL). Poniżej przedstawiono narzędzia użyte do realizacji tego projektu.

2.4.1 Python

Python został użyty jako główny język programowania po stronie backendu ze względu na swoją czytelność, elastyczność oraz bogaty ekosystem bibliotek. W projekcie posłużył do zaimplementowania warstwy serwerowej aplikacji, obsługi danych użytkownika, integracji ze skryptami do generowania sekwencji DNA oraz budowy REST API. Język ten doskonale sprawdził się w połączeniu z frameworkiem Django.

2.4.2 Django REST Framework

Django REST Framework (DRF) stanowi rozszerzenie frameworka Django, umożliwiające tworzenie interfejsów programistycznych zgodnych z architekturą REST. W niniejszym projekcie wykorzystano go do:

22:1677891584

- implementacji mechanizmów obsługujących żądania użytkowników, takich jak tworzenie, edytowanie i pobieranie danych sekwencyjnych,
- integracji z relacyjną bazą danych oraz istniejącymi modułami skryptowymi (`simple_tag.py`, `bits.py`),
- udostępnienia punktów końcowych (endpointów), z których korzysta część kliencka aplikacji.

Wybór DRF wynikał z jego szerokiego zastosowania w projektach webowych, dostępności rzetelnej dokumentacji oraz możliwości łatwej integracji z mechanizmami uwierzytelniania użytkowników.

REST API

API (**A**pplication **P**rogramming **I**nterface), to interfejs programowania aplikacji. Pozwala na komunikację pomiędzy dwoma aplikacjami. API używane jest pomiędzy komunikacją z aplikacją a serwerem sieciowym. Użytkownik wysyła żądanie (ang. request) i otrzymuje odpowiedź (ang. response) od serwera, odpowiedzią może być zarówno czynność wykonywana przez serwer, bądź też zwracane dane w np. formacie JSON [18].

Największą zaletą REST API (**RE**presentational **S**tate **T**ransfer), jest komunikacja z serwerem za pomocą sieciowych żądań HTTP. Przykładowe operacje, jakie są możliwe do wykonania, to [19]:

- **GET** – żądanie GET stosowane jest w celach pobrania danych z serwera do odczytu. Operacja ta nie przesyła żadnych danych, jedynie je pobiera. Gdy operacja się powiodła pomyślnie, zwracany jest kod statusu HTTP 200 (OK), a jeżeli nie jest w stanie pobrać danych, bądź takie nie istnieją, zwracany jest kod błędu, HTTP 404 (NOT FOUND).
- **POST** – żądanie POST służy do wysyłania danych i tworzenia nowych rekordów w bazie. Operacja POST jest w stanie przesłać wszelkie dane w formacie, jaki został określony na serwerze. W przypadku poprawnego żądania zostaje zwrócony kod statusu HTTP 201 (CREATED), a podczas błędu dostajemy kod HTTP 409 (CONFLICT) – jeżeli format jest zły, bądź identyczne dane już są zapisane w bazie i ponowny POST powoduje konflikt.
- **PUT** – metoda do modyfikacji danych na serwerze. Jest ona używana w celach aktualizacji całości danego rekordu, jeżeli dane do aktualizacji nie pasują do żądania, zostaną one utworzone. Zwraca kody statusu takie same jak wspomniana wyżej metoda GET, w przypadku sukcesu HTTP 200 (OK), błędu HTTP 404 (NOT FOUND).

23:8235260270

- **PATCH** – jest bardzo podobna do PUT, poza jednym wyjątkiem, nie tworzy nowych zawartości na serwerze i jest w stanie zmodyfikować ich część. Zwraca również HTTP 200 (OK) oraz HTTP 404 (NOT FOUND) w zależności od sukcesu.

- **DELETE** – żądanie DELETE służy do usuwania danych z serwera z określonych tabel. Gdy operacja się powiedzie, zwróci kod HTTP 200 (OK), w przypadku gdy dane nie istnieją -– HTTP 404 (NOT FOUND), a gdy pod konkretnym adresem nie istnieją wcale, otrzymuje się kod HTTP 204 (NO CONTENT).

Żądanie może zawierać w sobie wszelkie informacje, takie jak – *header*, przykładowo, klucz API/dane autoryzacyjne (ang. authentication data), *operacje* – informację o typie żądania sieciowego, *end-point* – adres URL do serwera, gdzie zostaną pobrane dane oraz *ciało* (ang. body).

Od serwera otrzymujemy odpowiedź, zawierając dane w wybranym formacie. Przykładowe żądanie, które przesyła zapytanie do serwera z operacją typu GET na end-point `/api/users` dostaje odpowiedź w formacie JSON (listing 2.1).

```
{
    "id": 1,
    "first_name": "Igor",
    "last_name": "Mazur",
    "username": "igormazur",
    "mail": "igor@gmail.com",
}
```

Listing 2.1: Przykładowy JSON

2.4.3 PostgreSQL

PostgreSQL został wybrany jako system zarządzania bazą danych ze względu na swoją stabilność, zgodność z relacyjnym modelem danych oraz możliwość przechowywania złożonych struktur, takich jak dane JSON. W aplikacji pełni rolę repozytorium dla:

- kont użytkowników,

- wygenerowanych sekwencji DNA,

- metadanych potrzebnych do ich organizacji, opisania i eksportu.

System ten zapewnia wysoką spójność danych i jest dobrze wspierany przez Django.

24:9555928450

2.4.4 JavaScript

JavaScript, jako język wykonywany po stronie klienta, został wykorzystany do budowy dynamicznych elementów interfejsu użytkownika. Dzięki niemu możliwe było tworzenie interakcji i aktualizacji danych w czasie rzeczywistym, bez konieczności przeładowywania całej strony. Język ten był szczególnie istotny przy współpracy z biblioteką React.

2.4.5 React

React.js to biblioteka frontendowa bazująca na komponentach, stworzona do budowy nowoczesnych interfejsów użytkownika. W aplikacji wykorzystano ją do:

- stworzenia przejrzystego i dynamicznego widoku dla użytkownika,
- zarządzania stanem aplikacji (np. lista sekwencji, podgląd struktury nici),
- komunikacji z backendem za pomocą żądań HTTP (np. pobieranie danych z REST API).

React umożliwia tworzenie modularnych komponentów, co znacząco przyspiesza rozwój i testowanie aplikacji.

2.4.6 Postman

Postman to narzędzie służące do testowania zapytań HTTP i interfejsów API. Został wykorzystany podczas budowy backendu do:

- testowania poprawności działania endpointów REST API,
- weryfikowania poprawności odpowiedzi zwracanych przez serwer,
- symulowania żądań wysyłanych przez użytkownika jeszcze przed implementacją warstwy frontendowej.

Dzięki Postmanowi możliwe było szybkie wychwycenie błędów logicznych w warstwie komunikacji serwera z klientem.

25:9638593553

Rozdział 3

Implementacja aplikacji webowej

W niniejszym rozdziale przedstawiono proces tworzenia aplikacji webowej służącej do zarządzania danymi zapisanymi w strukturze DNA. Prace nad systemem zostały podzielone na etapy zgodne z logiką tworzenia aplikacji webowej, obejmujące zarówno warstwę serwerową (backend), jak i warstwę prezentacyjną (frontend).

Każdy etap został zaplanowany i zrealizowany w sposób sekwencyjny, tak aby zapewnić stabilność systemu i możliwość testowania jego poszczególnych komponentów. Ogólny harmonogram działań przedstawia się następująco:

1. **Przygotowanie środowiska do pracy** – konfiguracja narzędzi programistycznych oraz utworzenie struktury projektu.

2. **Programowanie warstwy backendowej** – stworzenie logiki działania po stronie serwera z wykorzystaniem Django. Budowa REST API, implementacja modeli, widoków, konfiguracja adresów URL oraz tzw. serializerów (komponentów odpowiedzialnych za konwersję danych między obiektami a formatem JSON).

3. **Testowanie interfejsu API** – weryfikacja poprawności działania backendu z użyciem narzędzia Postman.

4. **Tworzenie warstwy frontendowej** – budowa interfejsu użytkownika przy użyciu biblioteki React.js.

W kolejnych podrozdziałach szczegółowo opisano każdy z powyższych etapów, pokazując w tym samym sposób realizacji oraz rozwiązania zastosowane podczas implementacji systemu.

26:8850040010

3.1 Przygotowanie środowiska do pracy

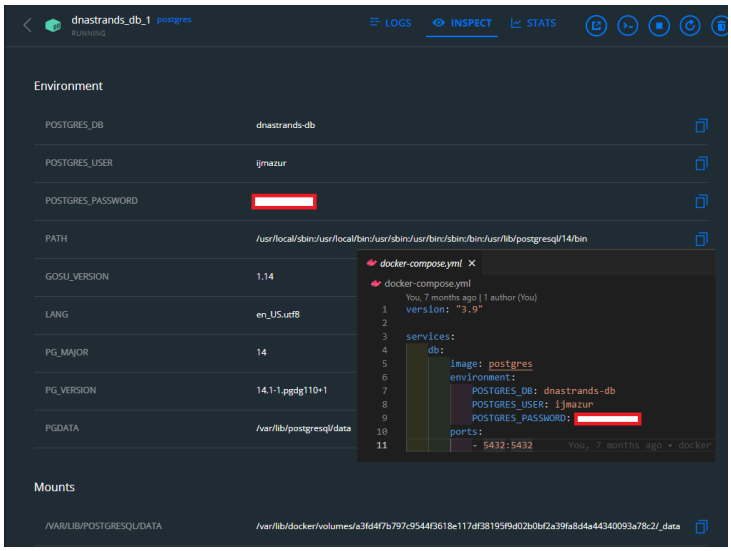
W pierwszym etapie implementacji aplikacji zostało przygotowane środowisko programistyczne. Do pracy wykorzystano darmowe środowisko programistyczne Visual Studio Code, opracowane przez firmę Microsoft, ze względu na jego szeroką kompatybilność z technologiami webowymi oraz bogaty ekosystem rozszerzeń.

W celu zainicjalizowania podstawowego szkieletu projektu zainstalowano: język programowania Python, środowisko uruchomieniowe Node.js oraz menedżer pakietów `npm`. Następnie, przy pomocy poleceń:

- `django-adminstartproject[nazwa_projektu]`,
- `npxcreate-react-app[nazwa_projektu]`,

zostały utworzone osobne katalogi projektowe dla aplikacji backendowej oraz frontendowej.

Po wykonaniu powyższych kroków projekt był gotowy do dalszej konfiguracji. Zdecydowano się na użycie systemu bazodanowego PostgreSQL, który został uruchomiony w kontenerze przy pomocy narzędzia Docker. W tym celu utworzono plik konfiguracyjny `docker-compose.yml`, w którym określono nazwę bazy danych, dane dostępowe oraz port komunikacyjny (pozostawiony na wartości domyślnej). Konfiguracja została przedstawiona na rysunku 3.1.



Rysunek 3.1: Ustawienia PostgreSQL w Dockerze oraz podgląd pliku `docker-compose.yml`.

Kolejnym etapem było dostosowanie ustawień w części serwerowej projektu. W pliku `settings.py` skonfigurowano parametry umożliwiające komunikację między warstwą backendową a aplikacją frontendową. W tym samym pliku doda-

27:1062331701

no również ustawienia związane z autoryzacją użytkowników przy użyciu biblioteki `djangorestframework-simplejwt`.

Biblioteka ta została wykorzystana do implementacji mechanizmu uwierzytelniania opartego na tokenach JWT (JSON Web Token), które generowane były przy każdym logowaniu użytkownika i pozostawały ważne przez określony czas. Szczegóły konfiguracji przedstawiono na listingu 3.1.

Tokeny JWT są podpisywane przy użyciu algorytmu HMAC z kluczem tajnym (`SECRET_KEY`) aplikacji, co zapewnia integralność i autentyczność przesyłanych danych. W przypadku konieczności unieważnienia tokenów, zastosowano mechanizm czarnej listy (token blacklisting), który pozwala na dezaktywację określonych tokenów przed upływem ich ważności.

Po zakończeniu konfiguracji szkieletu aplikacji webowej i po upewnieniu się, że połączenia z bazą danych działają kolejnym krokiem była implenetacja warsty serwerowej.

```
AUTH_USER_MODEL = 'core.MainAppUser'
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'ROTATE_REFRESH_TOKENS': False,
    'BLACKLIST_AFTER_ROTATION': True,
    'UPDATE_LAST_LOGIN': False,

    'ALGORITHM': 'HS256',
    'SIGNING_KEY': SECRET_KEY,
    'VERIFYING_KEY': None,
    'AUDIENCE': None,
    'ISSUER': None,

    'AUTH_HEADER_TYPES': ('Bearer',),
    'AUTH_HEADER_NAME': 'HTTP_AUTHORIZATION',
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',

    'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',),
    'TOKEN_TYPE_CLAIM': 'token_type',

    'JTI_CLAIM': 'jti',

    'SLIDING_TOKEN_REFRESH_EXP_CLAIM': 'refresh_exp',
    'SLIDING_TOKEN_LIFETIME': timedelta(minutes=60),
    'SLIDING_TOKEN_REFRESH_LIFETIME': timedelta(days=1),
```

28:9195728090

```
}
```

Listing 3.1: Ustawienia Simple JWT.

3.2 Programowanie warstwy backendowej

Budowa bazy API przy użyciu Django REST

Projektowanie interfejsu REST API rozpoczęto od zaimplementowania podstawowej funkcjonalności przeznaczonej dla użytkowników niezalogowanych – rejestracji oraz logowania. Punkt końcowy `/user` umożliwiał obsługę żądań HTTP typu `POST` (utworzenie konta) oraz `GET` (lista zarejestrowanych użytkowników). Funkcjonalność ta została w dalszej części aplikacji rozszerzona o operacje na danych związanych z sekwencjami DNA.

Podstawę warstwy serwerowej stanowiły modele danych, zdefiniowane w pliku `models.py`. Utworzono dwa główne modele: `MainAppUser` oraz `Tag`, z których każdy odpowiadał osobnej tabeli w bazie danych. Model użytkownika przechowywał dane rejestracyjne, natomiast model `Tag` reprezentował pojedynczą sekwencję DNA wraz z jej strukturą oraz właścicielem.

```
class MainAppUser(AbstractBaseUser):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    username = models.CharField(max_length=30, unique=True)
    password = models.CharField(max_length=255)
    mail = models.EmailField(max_length=254, unique=True)
    objects = UserManager()
    USERNAME_FIELD = 'username'

class Tag(models.Model):
    owner = models.ForeignKey(MainAppUser, on_delete = models.CASCADE, null =
        ↪ True, blank = True)
    secret = models.CharField(null=True, blank=True, max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)
    primer_1 = models.TextField()
    primer_2 = models.TextField()
    primer_3 = models.TextField()
    scheme = models.TextField()
    f1 = models.TextField()
    f2 = models.TextField()
    f3 = models.TextField()
    f1_f2 = models.TextField()
```

29:4277335687

```
    strand = models.TextField()

    def __str__(self):
        return self.strand
```

Listing 3.2: Modele z `models.py`.

W modelach zastosowano pola takie jak:

- `CharField` – do przechowywania tekstów o określonej długości, np. nazwy użytkownika,
- `EmailField` – do przechowywania unikalnego adresu e-mail z walidacją składni,
- `TextField` – do zapisu dłuższych ciągów danych (np. sekwencji DNA),
- `DateTimeField` – do automatycznego zapisu daty utworzenia rekordu,
- `ForeignKey` – do tworzenia relacji wiele-do-jednego między tagiem a użytkownikiem.

Widoki odpowiedzialne za obsługę żądań sieciowych zostały zdefiniowane w pliku `views.py`. Przykładowa klasa `TagViewSet`, przedstawiona na listingu 3.3, odpowiadała za generowanie, pobieranie i usuwanie sekwencji DNA. Wykorzystywała model `Tag` oraz serializator `TagSerializer`, a dane były generowane przy pomocy skryptu `simple_tag.py`.

```
class TagViewSet(APIView):
    def get(self, request, pk=None):
        if pk:
            tag = get_object_or_404(Tag.objects.all(), pk=pk)
            serializer = TagSerializer(tag)
            return Response({"Tag": serializer.data})
        be_data = simple_tag.generate()
        be_data['owner'] = request.user.id
        serializer = TagSerializer(data = be_data)
        if serializer.is_valid(raise_exception=True):
            serializer.save()

        tags = Tag.objects.all()
        serializer = TagSerializer(tags, many=True)
        return Response({"Tags": serializer.data})

    def post(self, request):
```

30:6115651643

```
        tag = request.data.get('Tag')
        serializer = TagSerializer(data=tag)
        if serializer.is_valid(raise_exception=True):
            tag_saved = serializer.save()
        return Response({"success": "Tag '{}' created
            ↪ successfully".format(tag_saved.verification_codes)})

    def delete(self, request, pk):
        tag = get_object_or_404(Tag.objects.all(), pk=pk)
        tag.delete()
        return Response({"message": "Tag with id `{}` has been
            ↪ deleted.".format(pk)},status=204)
```

Listing 3.3: Przykładowy Widok z `views.py`, obsługujący żądania sieciowe z URLu `/simple-tag/`.

Serializatory, zdefiniowane w pliku `serializers.py`, odpowiadały za przekształcanie danych modeli do formatu JSON oraz ich walidację. Poniżej przedstawiono fragment serializatora użytkownika:

```
class MainUserSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        password = validated_data.pop('password', None)
        instance = self.Meta.model(**validated_data)
        if password is not None:
            instance.set_password(password)
        instance.save()
        return instance

    class Meta:
        model = MainUser
        fields = '__all__'
        extra_kwargs = {
            'password': {'write_only': True}
        }
```

Listing 3.4: Plik `serializers.py`.

W pliku `urls.py` zarejestrowano ścieżki prowadzące do odpowiednich widoków, co umożliwiło ich wywoływanie przez żądania HTTP. Przykładowa konfiguracja znajduje się na listingu 3.5.

31:6089395939

```
router = routers.DefaultRouter()
router.register(r'user', views.MainUserViewSet)

urlpatterns = [
    path('', include(router.urls)),
    path('api/token/', TokenObtainPairView.as_view(),
        ↪ name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(),
        ↪ name='token_refresh'),
    path('api/token/user/', views.UserInfoView.as_view({'get':
        ↪ 'get_user_data'})),
    path('simple-tag/', views.TagViewSet.as_view(), name='generated_tag'),
    path('simple-tag/my-tags/', views.OwnerView.as_view()),
    path('simple-tag/my-tags/<int:pk>/', views.TagViewSet.as_view()),
    path('generate-bits/', views.BitViewSet.as_view(), name='generated_bits'),
    path('generate-bits/my-bits', views.BitOwnerView.as_view()),
    path('generate-bits/my-bits/<int:pk>', views.BitViewSet.as_view()),
]
```

Listing 3.5: Konfiguracja ścieżek z `urls.py`.

W powyższy sposób zrealizowano rdzeń warstwy backendowej aplikacji, łącząc modele danych z widokami, serializatorami oraz systemem routingu, co umożliwiło dalsze testowanie i integrację z interfejsem użytkownika.

3.3 Testowanie interfejsu API

Przed przystąpieniem do implementacji warstwy frontendowej przeprowadzono testy interfejsu API w celu weryfikacji poprawności działania poszczególnych punktów końcowych (endpointów) oraz ich zgodności z założeniami funkcjonalnymi. Testy miały również na celu upewnienie się, że backend prawidłowo przetwarza żądania HTTP, obsługuje sesje użytkowników oraz udostępnia odpowiednie dane.

W procesie testowym wykorzystano narzędzie Postman, umożliwiające wysyłanie żądań HTTP oraz analizę odpowiedzi serwera. W ramach testów sprawdzono m.in.:

- proces rejestracji i logowania użytkowników (uwierzytelnianie przy użyciu tokenów JWT),
- tworzenie, pobieranie, aktualizację i usuwanie danych (operacje CRUD),
- poprawność reakcji serwera na błędne lub niekompletne dane (walidacja i przypadki brzegowe).

32:1006882815

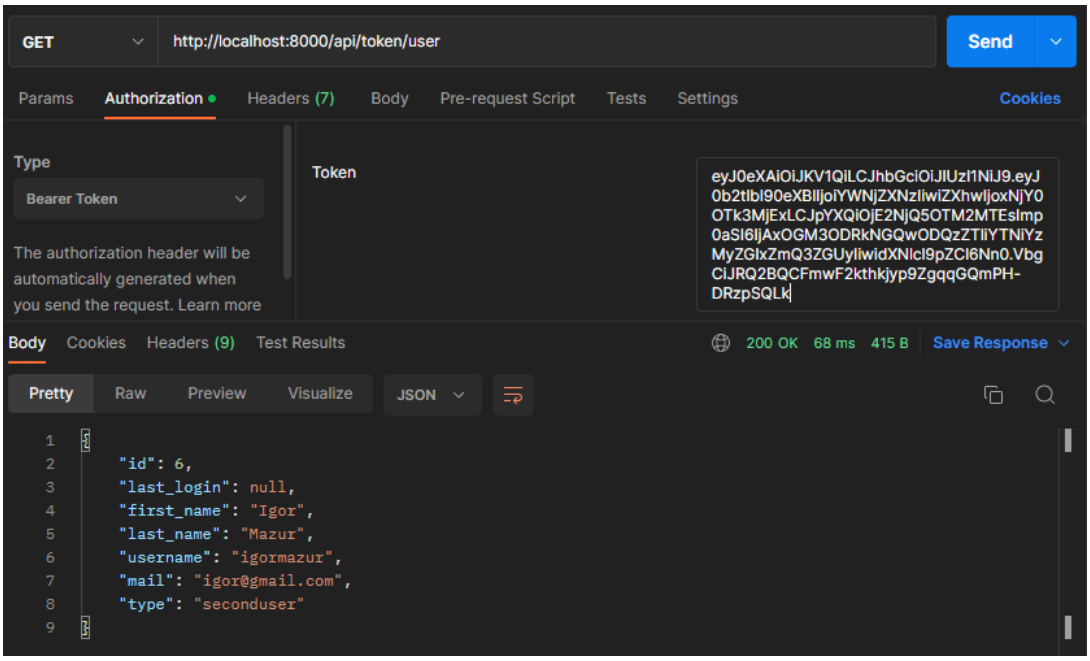
Ponadto, zweryfikowano strukturę bazy danych, która została oparta na relacyjnym modelu i przechowuje informacje o użytkownikach oraz przypisanych im sekwencjach DNA. Dane te są dostępne za pośrednictwem odpowiednich punktów API i mogą być edytowane, przeszukiwane oraz usuwane.

Po poprawnej konfiguracji uwierzytelniania JWT, udostępnione zostały trzy podstawowe adresy: `/api/token/`, `/api/token/user/` oraz `/api/token/refresh/`. Wysłanie żądania POST na adres `/api/token/` z danymi uwierzytelniającymi użytkownika w formacie JSON skutkowało otrzymaniem pary tokenów: dostępowego i odświeżającego (rysunek 3.2).



Rysunek 3.2: Odpowiedź z `/api/token/` – token dostępowy i odświeżający.

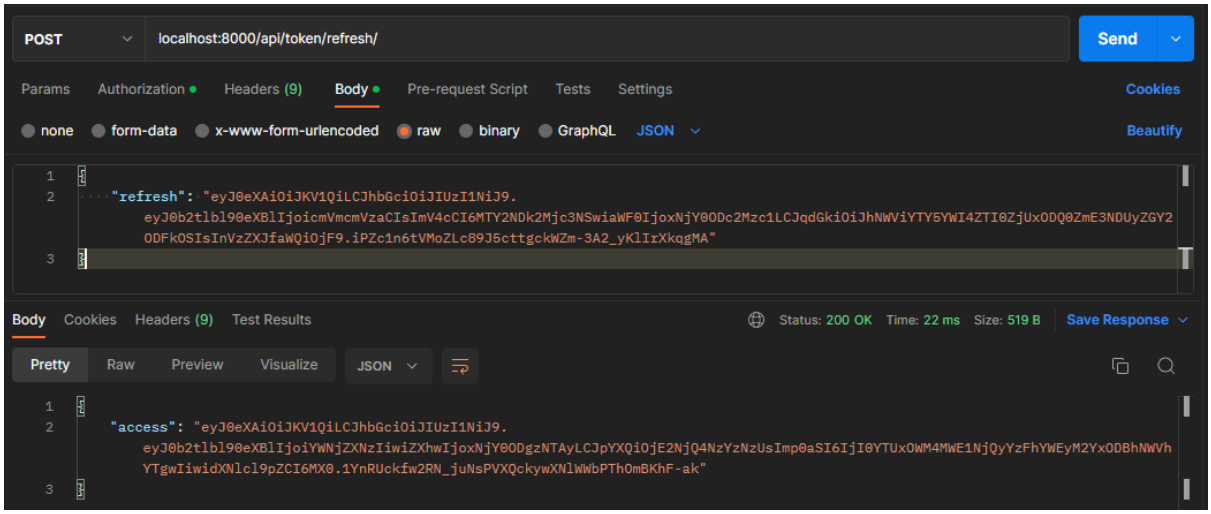
Zgodnie z konfiguracją przedstawioną w listingu 3.1, token dostępowy był aktywny przez 60 minut (`ACCESS_TOKEN_LIFETIME`), natomiast token odświeżający przez 24 godziny (`REFRESH_TOKEN_LIFETIME`). W przypadku braku aktywności użytkownika przez określony czas wymagane było ponowne logowanie lub przesłanie tokena odświeżającego na adres `/api/token/refresh/`.



Rysunek 3.3: Odpowiedź na zapytanie GET do `/api/token/user/`.

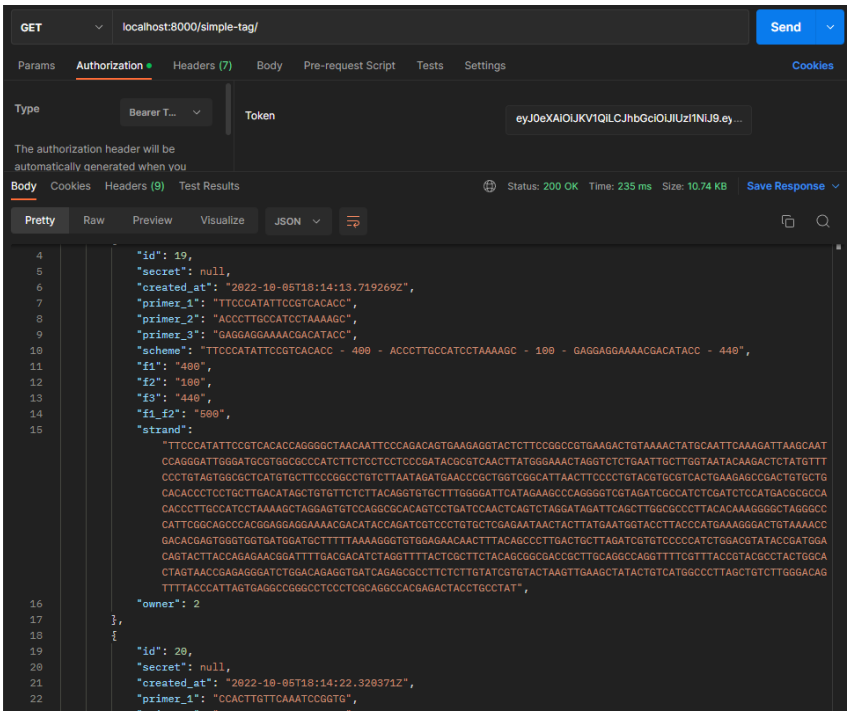
33:5202400490

Na rysunku 3.3 przedstawiono odpowiedź na zapytanie GET do punktu `/api/token/user/`, które zwracało dane aktualnie zalogowanego użytkownika. Konieczne było uprzednie ustawienie nagłówka HTTP `Authorization` z wartością tokena JWT poprzedzoną słowem 'Bearer'.



Rysunek 3.4: Odpowiedź z `/api/token/refresh/` - nowy token dostępowy.

Przesłanie żądania POST z tokenem odświeżającym na adres `/api/token/refresh/` poskutkowało otrzymaniem nowego tokena dostępowego, jak zostało to zaprezentowane na rysunku 3.4.



Rysunek 3.5: Odpowiedź GET z `/simple-tag/` – lista sekwencji DNA.

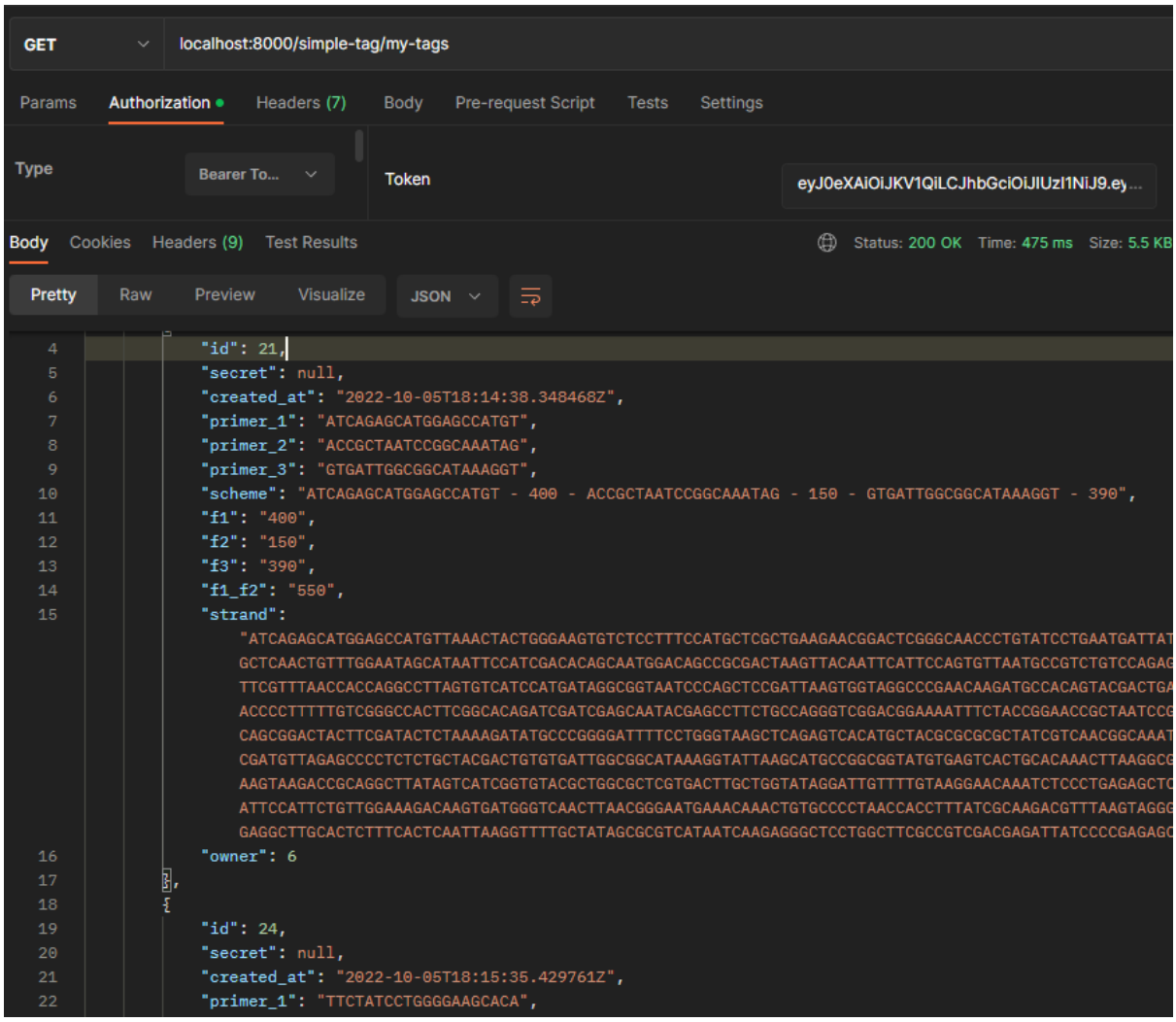
Żądanie GET wysłane na adres `/simple-tag/` zwracało listę wszystkich zarejestrowa-

34:9898864044

nych sekwencji DNA. Każdy rekord zawierał informację o użytkowniku, do którego dany tag był przypisany (pole `owner_id`). Generowanie tagów odbywało się z wykorzystaniem skryptu `simple_tag.py`.

Aby uzyskać tylko własne rekordy, należało skorzystać z punktu `/simple-tag/my-tags/` (rysunek 3.6). Możliwe było również pobranie konkretnej sekwencji na podstawie jej identyfikatora, a także jej usunięcie za pomocą metody DELETE.

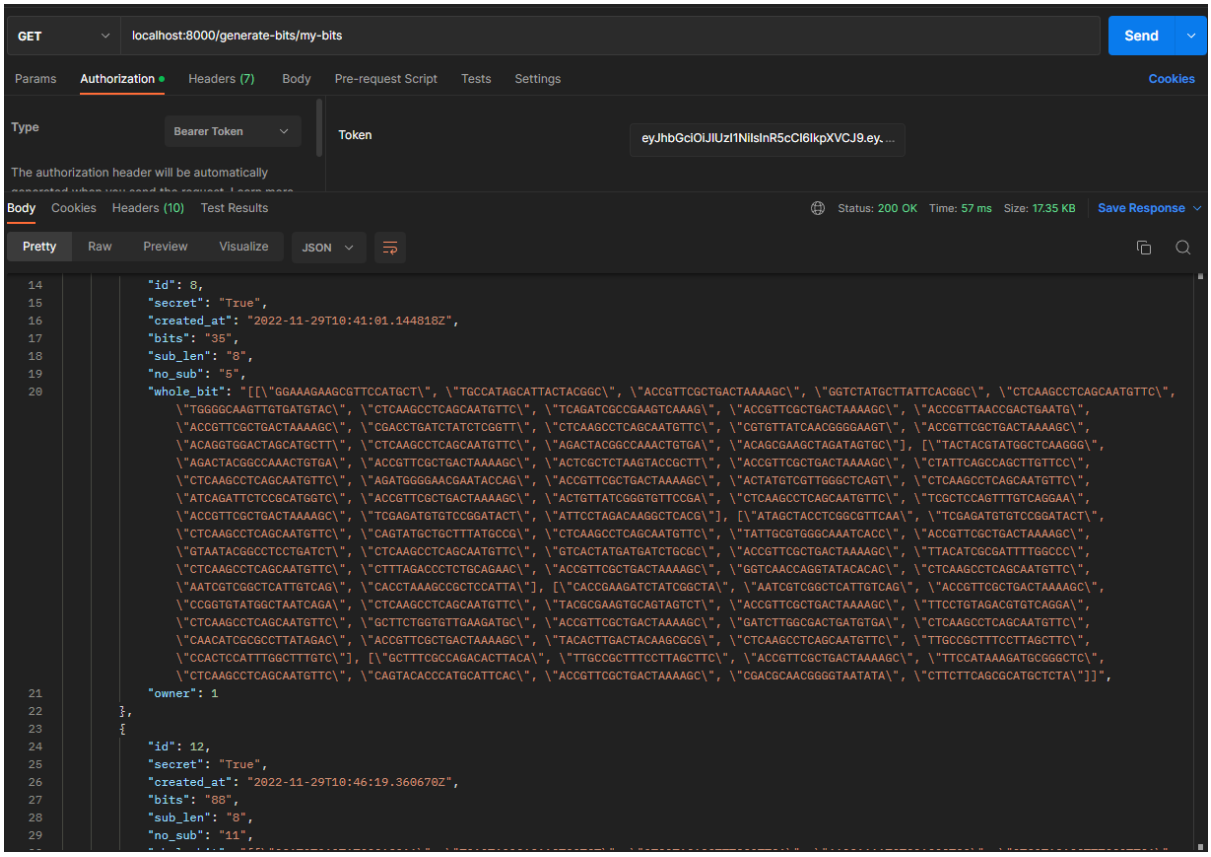
Pozwala to również na łatwą obsługę metody DELETE. Istnieje możliwość odpytania tego URL równocześnie używając GETa jak i DELETE.



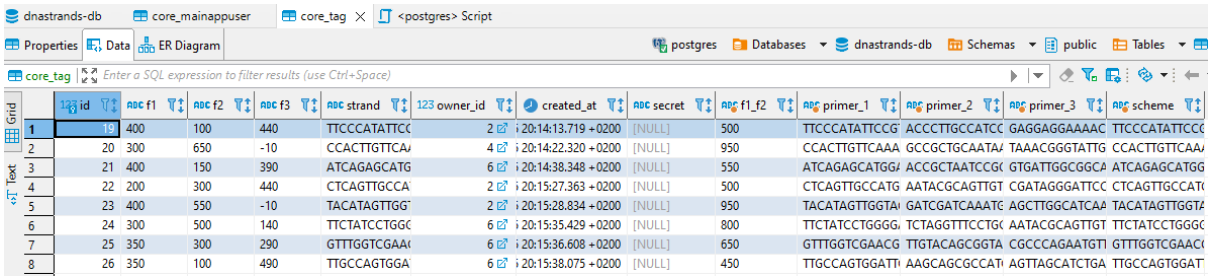
Rysunek 3.6: Lista tagów zalogowanego użytkownika – `/simple-tag/my-tags/`.

Podobna funkcjonalność została zaimplementowana dla danych generowanych przez skrypt `bits.py`. Żądanie GET wysłane na adres `/generate-bits/my-bits/` zwracało sekwencje binarne przypisane do aktualnego użytkownika (rysunek 3.7).

35:1095953787

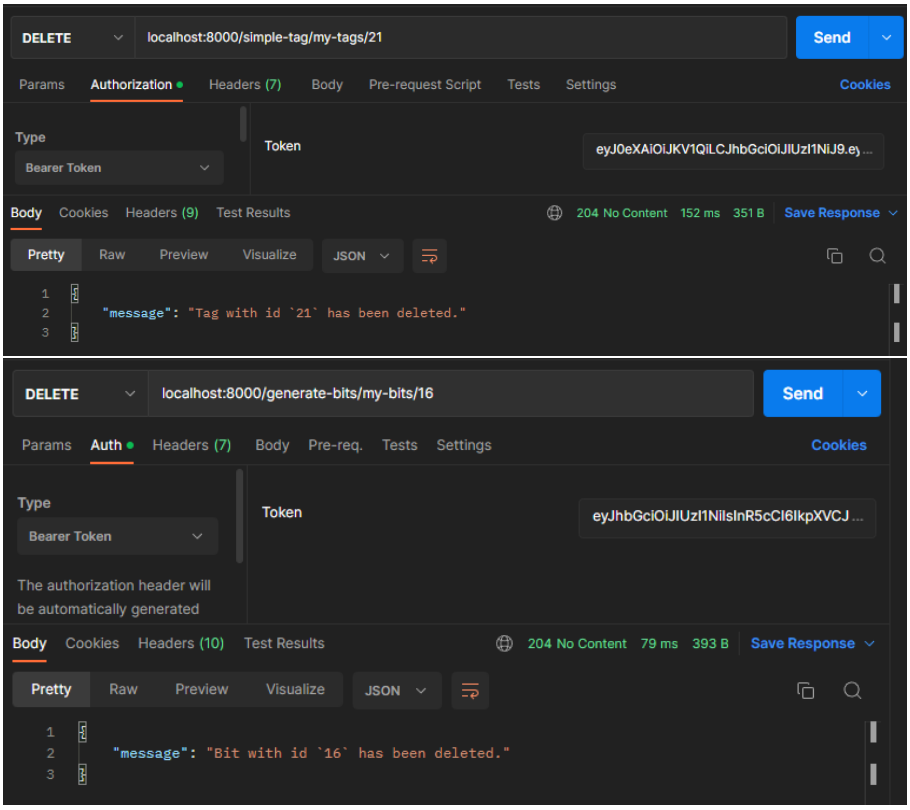


Rysunek 3.7: Lista wygenerowanych sekwencji kodujących bity – `/generate-bits/my-bits/`.



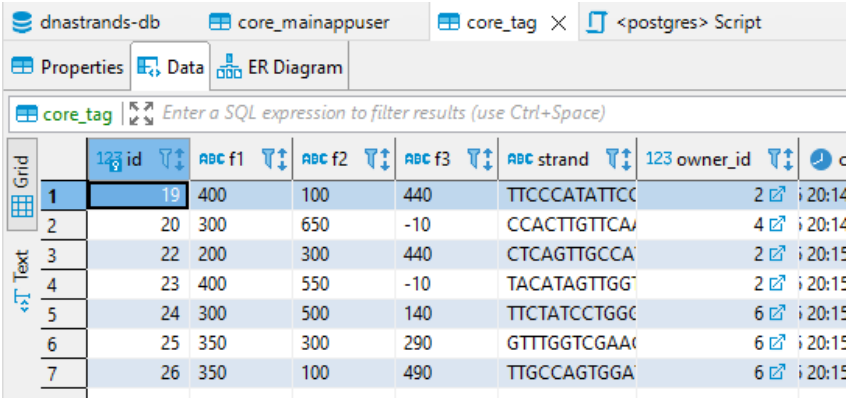
Rysunek 3.8: Podgląd tagów w bazie.

Przed wykonaniem operacji usuwania, dane były widoczne w bazie danych, co zilustrowano na rysunku 3.8.



Rysunek 3.9: Odpowiedzi po wykonaniu żądań DELETE na adresy `/simple-tag/my-tags/21/` oraz `/generate-bits/my-bits/16/`.

Po wykonaniu operacji DELETE rekordy zostały poprawnie usunięte, co potwierdza zarówno odpowiedź serwera (rysunek 3.9), jak i bezpośredni podgląd stanu bazy danych.



Rysunek 3.10: Podgląd bazy po prawidłowym wykonaniu operacji DELETE.

W celach weryfikacji usunięcia rekordu z bazy odświeżono tabele posiadające dane o tagach. Używając programu DBeaver zweryfikowano, że żądanie powiodło się i rekord o ID 21 został usunięty z bazy (rysunek 3.10).

37:7731038371

3.4 Tworzenie warstwy frontendowej

Implementacja warstwy wizualnej przy użyciu React

Warstwa frontendowa aplikacji została zaimplementowana przy użyciu biblioteki React. Jej celem było zapewnienie użytkownikowi przejrzystego, interaktywnego interfejsu umożliwiającego rejestrację, logowanie oraz zarządzanie generowanymi sekwencjami DNA.

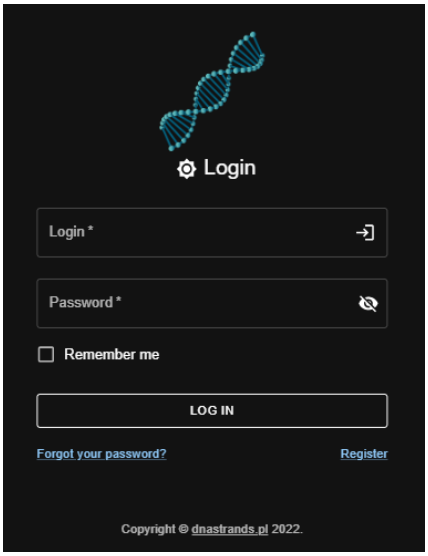
Na początkowym etapie zdefiniowano strukturę routingu aplikacji z wykorzystaniem biblioteki `react-router`, umożliwiającej dynamiczne przełączanie się pomiędzy widokami komponentów. Listing 3.6 prezentuje fragment pliku `App.js`, zawierający główne ścieżki URL.

```
return (
    <ColorModeContext.Provider value={colorMode}>
      <ThemeProvider theme={theme}>
        <Router >
          <Routes>
            <Route path="/" element={<Navigate replace to='/login' />} />
            <Route path='/register' element={<Register button={<ThemeButton
                ↪ />}/>}/>
            <Route path="/login" element={<Login button={<ThemeButton />}/>} />
            <Route path="/main" element={<Dashboard button={<ThemeButton
                ↪ />}/>} />
            <Route path="/profile" element={<Profile button={<ThemeButton
                ↪ />}/>} />
            <Route path="/history" element={<History button={<ThemeButton
                ↪ />}/>} />
            <Route path="/core" element={<Core button={<ThemeButton />}/>} />
            <Route path="/test" element={<Test button={<ThemeButton />}/>} />
            <Route path="/results" element={<Results button={<ThemeButton
                ↪ />}/>} />
            <Route path="/history/:id" element={<MyTags button={<ThemeButton
                ↪ />}/>}/>
            <Route path="/bits/:id" element={<Bits button={<ThemeButton />}/>}
                ↪ />
          </Routes>
        </Router >
      </ThemeProvider>
    </ColorModeContext.Provider>
  );
```

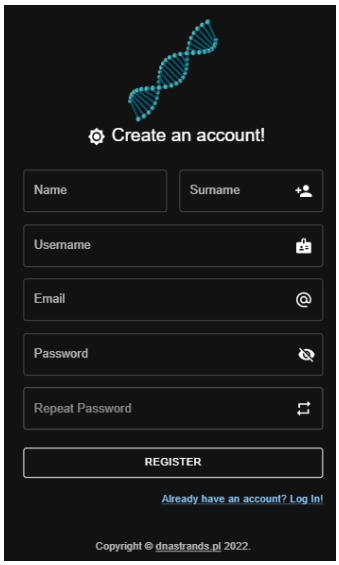
Listing 3.6: Podgląd pliku `App.js`, w którym znajdują się ścieżki.

38:9024336009

Pierwszymi komponentami, które zostały zaimplementowane, były widoki logowania i rejestracji (rysunek 3.11).



(a) Widok ekranu logowania



(b) Widok ekranu rejestracji

Rysunek 3.11: Podgląd pierwszych widoków.

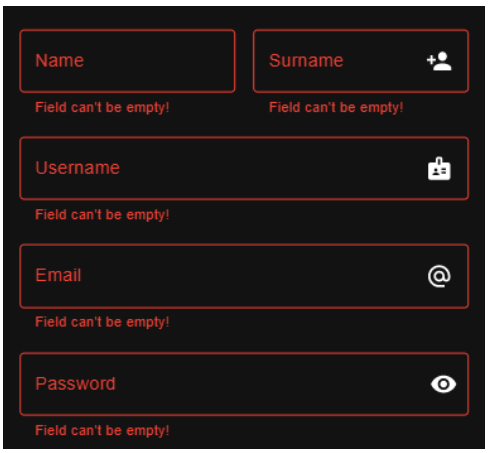
Ponieważ frontend aplikacji komunikował się z backendem poprzez żądania HTTP, przygotowano logikę odpowiedzialną za ich obsługę z wykorzystaniem biblioteki `axios`. W pliku `auth.service.js` zaimplementowano metody realizujące zapytania typu POST, wykorzystywane podczas rejestracji i logowania. Parametry wprowadzane przez użytkownika do formularzy zostały przekazywane do odpowiednich funkcji serwisowych (listing 3.7).

```
const handleSubmit = (event) => {
    event.preventDefault();
    const data = new FormData(event.currentTarget);
    const username = data.get('email');
    const password = data.get('password');

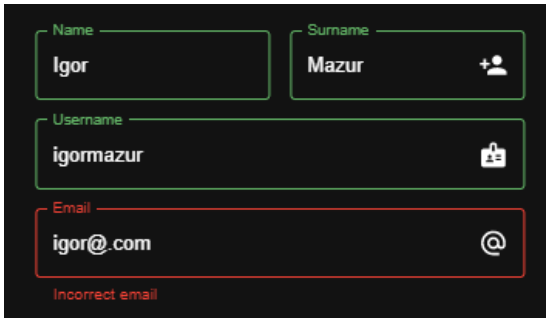
    authService.login(username, password, data).then(
      (success) => { if (data === 'user') { navigate('/main'); } },
      (error) => { alert('You entered wrong username or password!'); }
    );
  };
```

Listing 3.7: Funkcja *handleSubmit*, odpowiada za wywołanie funkcji *authService.login* z danymi pobranymi z forumlarza logowania. W przypadku powodzenia, kieruje ona użytkownika do widoku (URL) `/main`.

39:1063916793

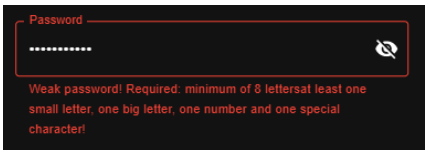


(a) Walidacja pustych pól

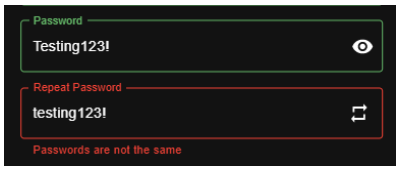


(b) Walidacja poprawnego formatu Email

Rysunek 3.12: Walidacja danych w oknie rejestracji.



(a) Walidacja silności hasła



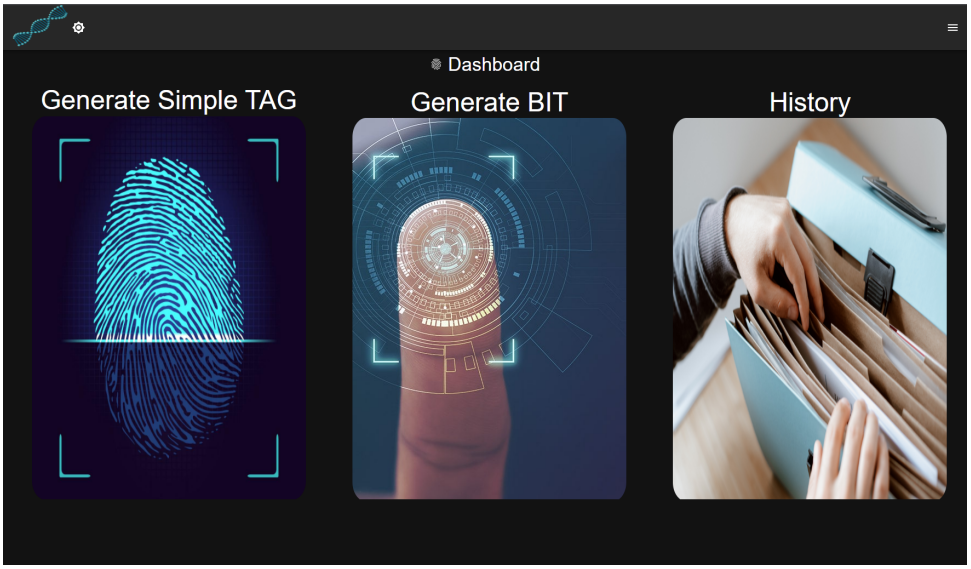
(b) Walidacja hasła z funkcją podejrzenia pola

Rysunek 3.13: Możliwy podgląd wpisanego hasła, po wciśnięciu guzika w polu *Password*.

Przed przesłaniem danych, formularze podlegały walidacji, obejmującej m.in. poprawność formatu e-maila, siłę hasła oraz kompletność pól (rysunki 3.12 i 3.13).

Po poprawnej autoryzacji użytkownik był przekierowywany do głównego panelu aplikacji (rysunek 3.14), gdzie uzyskiwał dostęp do trzech głównych funkcjonalności: generowania tagów, generowania sekwencji kodujących bity oraz przeglądania historii wygenerowanych sekwencji.

40:2726890006



Rysunek 3.14: Panel główny.

Powyższe funkcje, umożliwiały dostęp do trzech kluczowych funkcji systemu:

- **Generate Simple TAG** – otwierało okno dialogowe umożliwiające wygenerowanie nowej sekwencji taga DNA,
- **Generate BIT Sequence** – uruchamiało formularz wymagający wprowadzenia danych wejściowych do wygenerowania sekwencji kodującej podany przez użytkownika ciąg bitów,
- **History** – umożliwiało przejście do widoku historii wcześniej utworzonych sekwencji.

Kod komponentu odpowiedzialnego za funkcjonalność **Generate Simple TAG**, przedstawiony na listingu 3.8

```
<Box
    sx={{
      display: 'flex', justifyContent: 'space-around', m: 1, flexDirection:
          ↪ 'column',
      alignItems: 'stretch', backgroundSize: 'cover', backgroundRepeat:
          ↪ 'no-repeat',
      width: '500px', height: '700px', borderRadius: '5%',
    }}>
    <Typography variant="h3" style={{ textAlign: 'center' }}> Generate Simple
        ↪ TAG </Typography>
    <div onMouseEnter={() => setIsShown2(true)} onMouseLeave={() =>
        ↪ setIsShown2(false)}>
```

41:1036321967

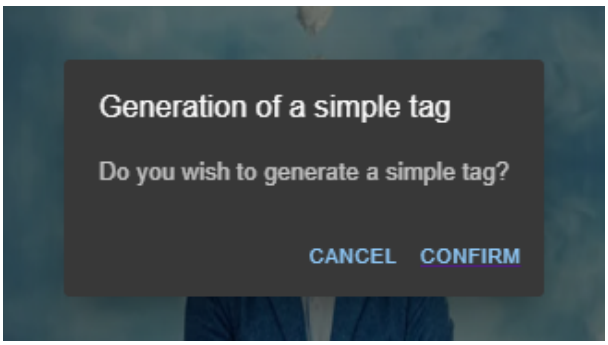
```
      <img src={require("../../images/fingerprint.jpg")} alt='Generate2'
          ↪ width={'100%'}
          style={{ borderRadius: '5%', opacity: isShown2 ? 0.75 : 1, width:
              ↪ '100%', height: '700px' }}
          onClick={handleClickOpen2} />
        <Dialog open={openDialog2} onClose={handleClose2} >
          <DialogTitle>{"Generation of a simple tag"}</DialogTitle>
          <DialogContent>
            <DialogContentText id="alert-dialog-slide-description">
              Do you wish to generate a simple tag?
            </DialogContentText>
          </DialogContent>
          <DialogActions>
            <Button onClick={handleClose2}>CANCEL</Button>
              <Link to="/history">
                <Button onClick={handleSimpleTag}>CONFIRM</Button>
              </Link>
          </DialogActions>
        </Dialog>
    </div>
</Box>
```

Listing 3.8: Fragment kodu z komponentu odpowiadającego za widok ekranu głównego z rysunku 3.14.

Zachowanie interaktywne obrazu zostało zrealizowane poprzez reakcję na zdarzenie najechania kursorem myszy – w momencie aktywacji, przezroczystość obrazu zmieniała się na 75%, co sygnalizowało możliwość interakcji. Kliknięcie powodowało ustawienie odpowiedniego stanu logicznego, co skutkowało otwarciem okna dialogowego (rysunek 3.15).

Po wyświetleniu okna dialogowego użytkownik mógł potwierdzić chęć wygenerowania taga. Kliknięcie przycisku `Confirm` inicjowało wywołanie funkcji `handleSimpleTag()`, która odpowiadała za wysłanie żądania HTTP do backendu. Po uzyskaniu odpowiedzi, użytkownik był automatycznie przekierowywany do widoku `/history`, gdzie mógł zobaczyć wygenerowaną sekwencję wraz z poprzednimi rekordami.

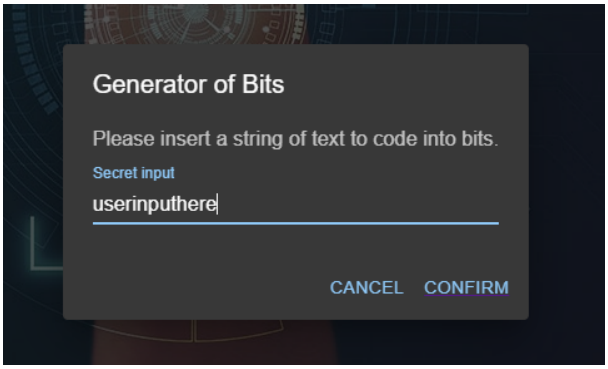
42:2790038467



Rysunek 3.15: Okno dialogowe, otwierane na środku ekranu.

Na rysunku 3.16 przedstawiono okno dialogowe, umożliwiające wprowadzenie danych wejściowych przez użytkownika. Wprowadzone informacje były następnie wykorzystywane przez skrypt `bits.py` do wygenerowania sekwencji DNA w oparciu o przesłane wartości.

Po zatwierdzeniu danych, aplikacja wysyłała żądanie HTTP typu POST z formularza frontendowego do warstwy serwerowej. Backend przetwarzał dane, generował wynik, a następnie w nastepnej kolejności odsyłał odpowiedź w formacie JSON. Uzyskana odpowiedź była wyświetlana użytkownikowi bezpośrednio w interfejsie.



Rysunek 3.16: Okno dialogowe oczekujące na dane wejściowe od użytkownika.

Funkcja przedstawiona na listingu 3.9 odpowiadała za obsługę kliknięcia przycisku potwierdzającego i inicjowała proces generowania sekwencji przez wywołanie metody `generateTag()` z klasy serwisowej.

```
const handleSimpleTag = () => {
  simpleTagService.generateTag();
};
```

Listing 3.9: Funkcja **handleSimpleTag**

Kod odpowiedzialny za wykonanie zapytania HTTP GET został umieszczony w klasie `SimpleTagService`. Wysyłane żądanie zawierało w nagłówku token autoryzacyjny JWT,

43:4280494297

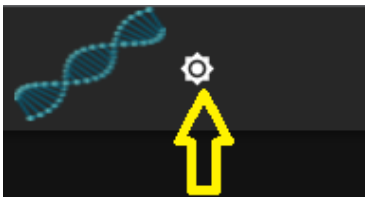
pozyskany z lokalnego magazynu przeglądarki. Odpowiedź serwera była zwracana jako dane typu JSON (listing 3.10).

```
class SimpleTagService {
  generateTag(){
    const config = defaultConfig;
    config.headers['Authorization'] = 'Bearer ' +
        ↪ localStorage.getItem('user');
    return axios.get(API_SIMPLETAG_URL, config)
      .then(response => response.data)
  }
}
```

Listing 3.10: Metoda realizująca żądanie do API w celu wygenerowania taga

Użytkownik miał możliwość zmiany motywu kolorystycznego interfejsu za pomocą ikony koła zębatego, widocznej na rysunku 3.17a. Preferencje dotyczące motywu były zapisywane w ciasteczkach (ang. cookies) – niewielkich plikach przechowywanych po stronie klienta, które przy każdym ponownym wejściu na stronę internetową były automatycznie odczytywane przez aplikację.

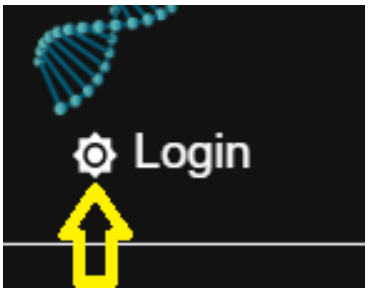
Dla użytkowników zalogowanych dostępne było również wysuwane menu nawigacyjne (rysunek 3.17b), umożliwiające szybki dostęp do podstawowych funkcjonalności systemu. Obraz nici DNA, umieszczony obok przycisku zmiany motywu, pełnił dodatkowo funkcję przycisku przekierowującego użytkownika z powrotem do głównego widoku aplikacji, znajdującego się pod adresem `/main`.



(a) Główna ikona zmiany motywu.



(b) Przycisk wysuwający boczne menu.



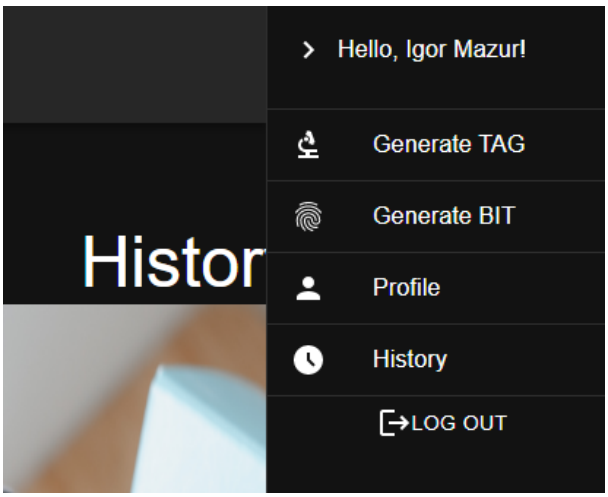
(c) Zmiana motywu podczas logowania.



(d) Zmiana motywu podczas rejestracji.

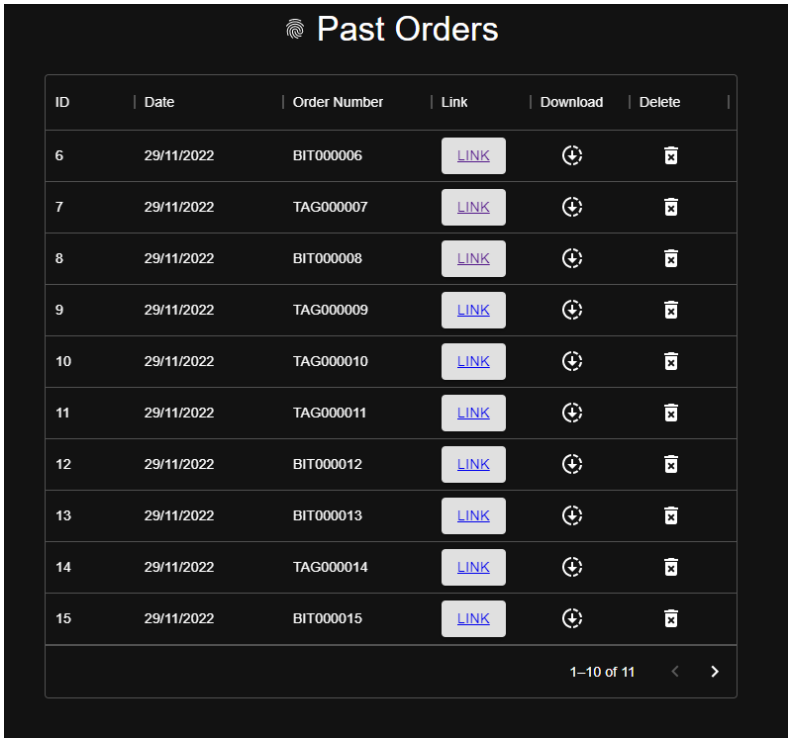
Rysunek 3.17: Dodatkowe przyciski ułatwiające, tzw. QoL (ang. quality of life).

44:5991739803



Rysunek 3.18: Wysunięte boczne menu nawigacyjne.

Menu nawigacyjne wysuwane z prawej krawędzi ekranu umożliwiało przechodzenie pomiędzy głównymi widokami aplikacji. Zostało ono zaprojektowane z myślą o łatwej i intuicyjnej obsłudze oraz dynamicznym pobieraniu danych użytkownika z serwera API.



Rysunek 3.19: Historia poprzednio wygenerowanych tagów.

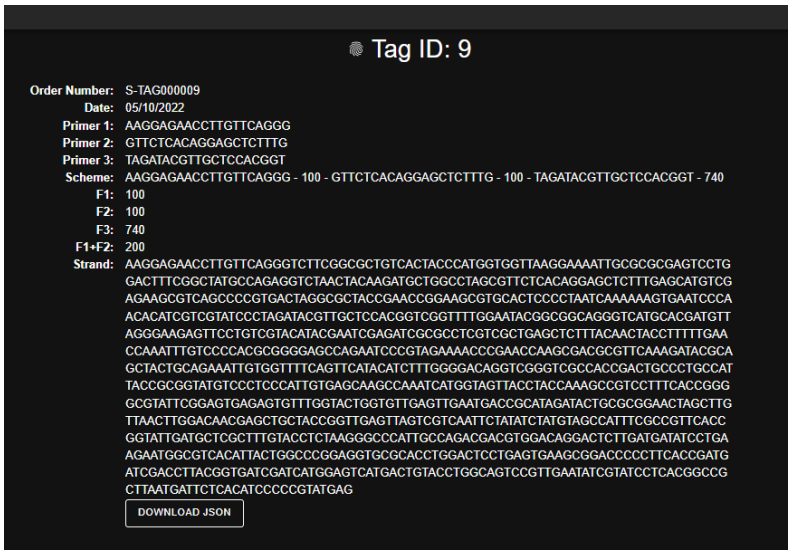
Widok dostępny pod adresem `/history` umożliwiał zalogowanemu użytkownikowi przeglądanie historii wygenerowanych tagów DNA (rysunek 3.19). W tabeli prezentowano podstawowe informacje, takie jak:

- `ID` – unikalny identyfikator taga lub sekwencji bitowej,

45:2688209977

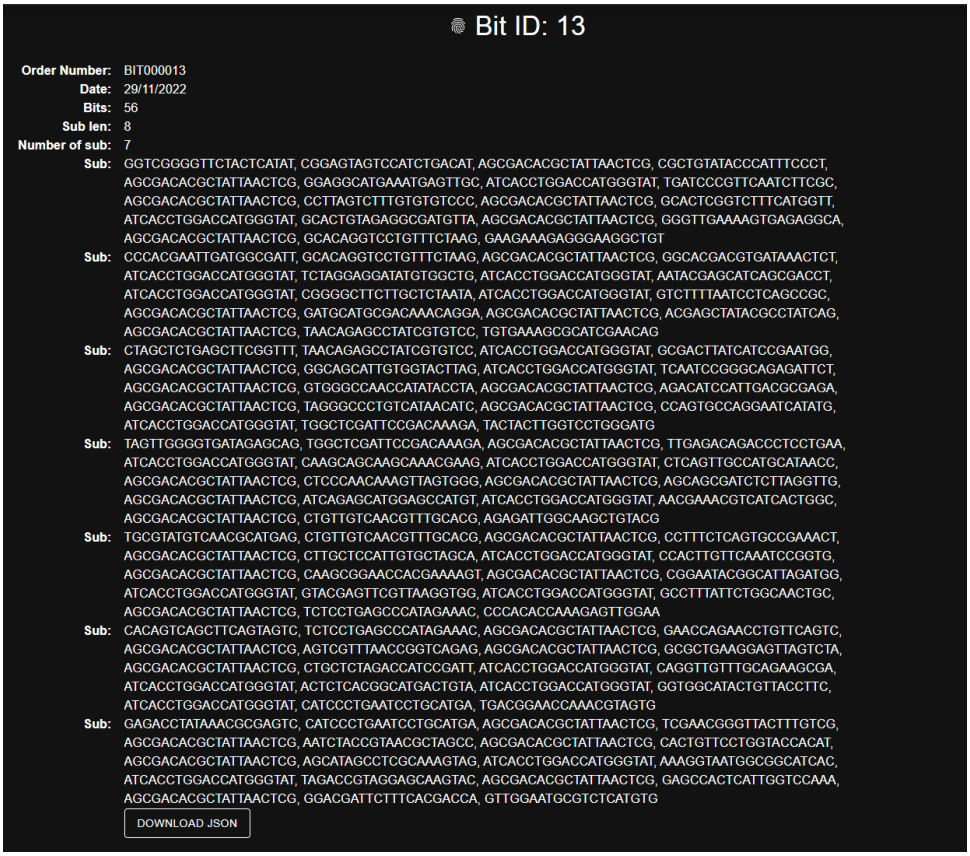
- `Date` – data utworzenia rekordu,
- `Order Number` – numer porządkowy nadany danej sekwencji, przy czym `BITXXXXXX` oznacza identyfikator sekwencji bitów, a `TAGXXXXXX` to identyfikator znacznika.
- `Link` – odnośnik umożliwiający przejście do widoku szczegółowego danego rekordu pod adresem `/history/{id}`,
- `Download` – przycisk umożliwiający pobranie danych w formacie `.json` (rysunek 3.22),
- `Delete` – opcja pozwalająca na usunięcie rekordu z bazy danych za pomocą żądania HTTP typu DELETE.

Kliknięcie przycisku `Link` powodowało przejście do szczegółowego widoku konkretnego taga (rysunek 3.20) lub sekwencji bitów (rysunek 3.21). Widok ten zawierał wszystkie informacje wygenerowane podczas tworzenia sekwencji i umożliwiał ich pobranie w formacie `.json`, analogicznie jak w widoku historii.



Rysunek 3.20: Szczegóły pojedynczego taga.

46:9895699069



Rysunek 3.21: Szczegóły dla sekwencji kodującej bity wprowadzone przez użytkownika.

Dane wyświetlane w widoku szczegółowym mogły zostać pobrane w formie pliku `.json`, generowanego automatycznie przez aplikację (rysunek 3.22). Plik ten zawierał pełną strukturę sekwencji i mógł zostać zapisany lokalnie przez użytkownika.



Rysunek 3.22: Automatycznie wygenerowany plik w formacie `.json`

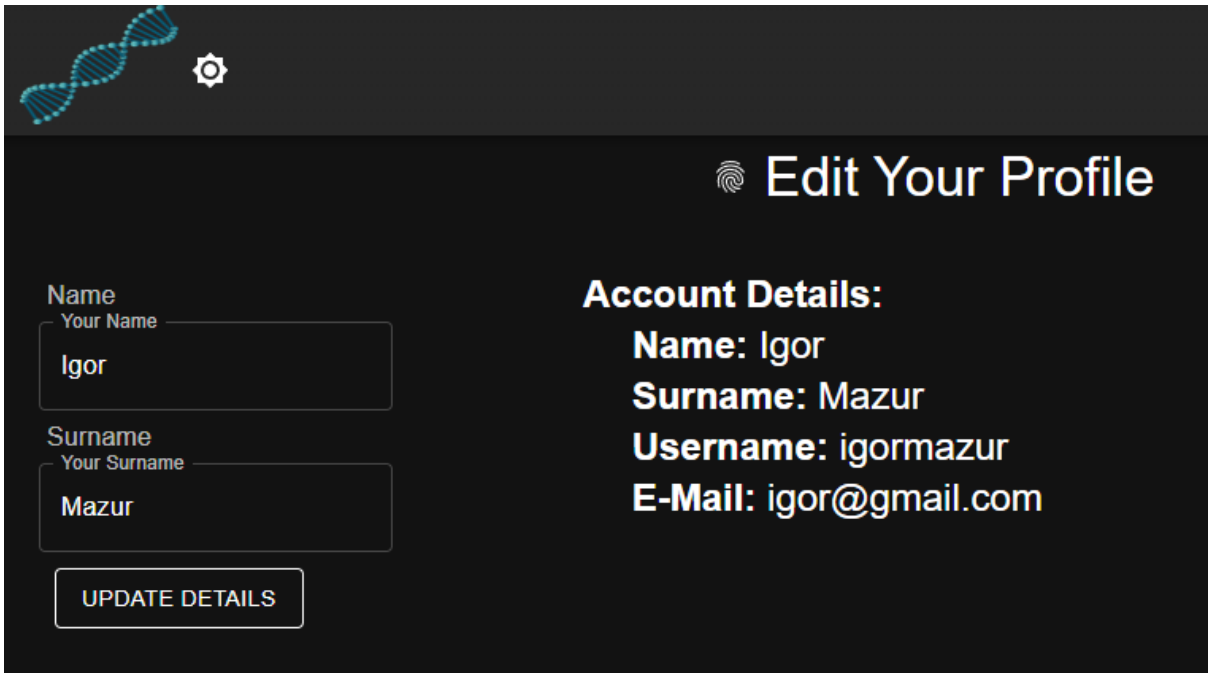
47:1613129958

Do obsługi funkcji pobierania danych w formacie `.json` wykorzystano bibliotekę `FileSaver.js`. Listing 3.11 przedstawia funkcję odpowiedzialną za eksport danych z widoku `/history` lub widoku szczegółowego konkretnego taga pod adresem `/history/{id}`.

```
const handleDownload = (event, cellValues) => {
  pastOrderService.getTagById(cellValues.id).then((response) => {
    const fileName = `${cellValues.row.orderNumber}.json`;
    const fileToSave = new Blob([JSON.stringify(response, undefined, 2)], {
      type: 'application/json'
    });
    saveAs(fileToSave, fileName);
  });
};
```

Listing 3.11: Fragment kodu odpowiedzialnego za pobieranie danych w formacie JSON.

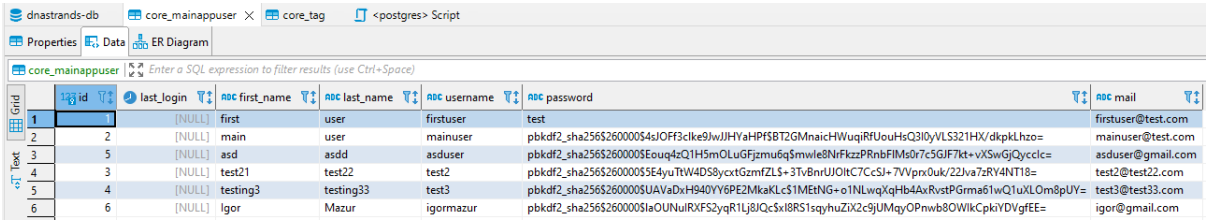
Funkcja ta była uruchamiana po kliknięciu odpowiedniego przycisku w interfejsie użytkownika, a jej działanie polegało na pobraniu danych dotyczących wskazanego taga i zapisaniu ich lokalnie jako pliku `.json` o nazwie odpowiadającej numerowi zamówienia (*Order Number*).



Rysunek 3.23: Profil użytkownika przed aktualizacją danych.

Na ekranie profilu użytkownika (rysunek 3.23) wyświetlane zostały wszystkie dane zalogowanego konta, pobrane bezpośrednio z bazy danych. Dane te były jednocześnie automatycznie uzupełniane w formularzu edycji, co pozwalało użytkownikowi wprowadzać zmiany bez konieczności ręcznego wpisywania wcześniejszych wartości.

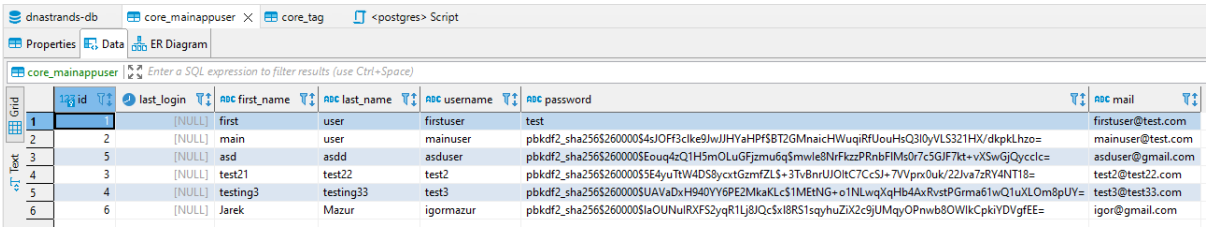
48:1479204860



	id	last_login	first_name	last_name	username	password	mail
1		[NULL]	first	user	firstuser	test	firstuser@test.com
2	2	[NULL]	main	user	mainuser	pbkdf2_sha256$260000$4sJOFf3clke9JwJJHYaHPf$BT2GMnaicHWuqiRfUouHsQ3l0yVLS321HX/dkpkLhzo=	mainuser@test.com
3	5	[NULL]	asd	asdd	asduser	pbkdf2_sha256$260000$Eouq4zQ1H5mOLuGFjzmu6q$mwle8NrFkzzPRnbFIMs0r7c5GJF7kt+vXSwGjQycclc=	asduser@gmail.com
4	3	[NULL]	test21	test22	test2	pbkdf2_sha256$260000$5E4yuTtW4DS8ycxtGzmfZL$+3TvBnrUJOltC7CcSJ+7VVprx0uk/22Jva7zRY4NT18=	test2@test22.com
5	4	[NULL]	testing3	testing33	test3	pbkdf2_sha256$260000$UAVaDxH940YY6PE2MkaKLc$1MEtNG+o1NLwqXqHb4AxRvstPGrma61wQ1uXLOm8pUY=	test3@test33.com
6	6	[NULL]	Igor	Mazur	igormazur	pbkdf2_sha256$260000$IaOUNulRXFS2yqR1Lj8JQc$xI8RS1sqyhuZiX2c9jUMqyOPnwb8OWIkCpkiYDVgfEE=	igor@gmail.com

Rysunek 3.24: Podgląd bazy danych przed edycją profilu.

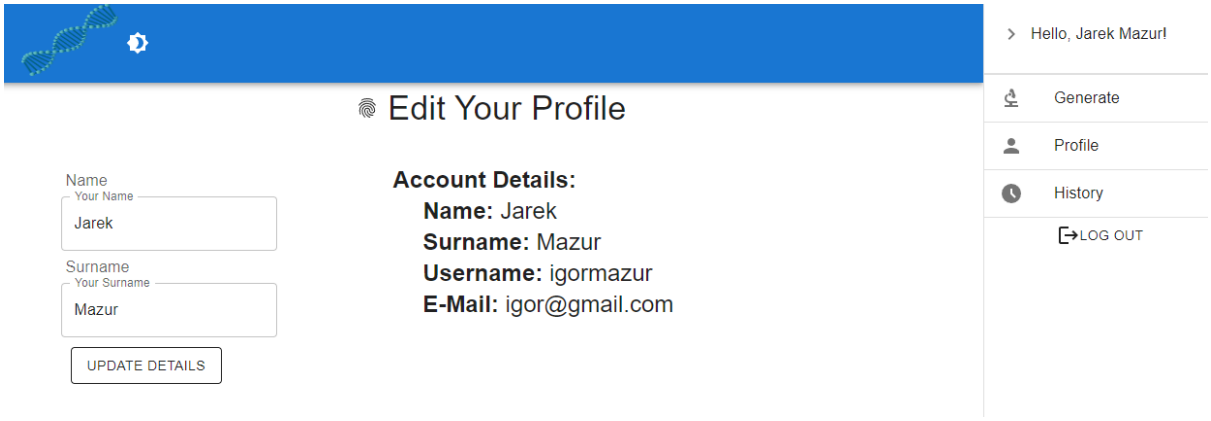
W przykładzie przedstawionym na rysunkach 3.24 i 3.25, zalogowany użytkownik o identyfikatorze ID 6 dokonał zmiany swojego imienia w polu `Name` z „Igor” na „Jarek”. Po zatwierdzeniu formularza aplikacja przesłała żądanie sieciowe do backendu, co skutkowało aktualizacją rekordu w bazie danych – dokładnie w kolumnie `first_name` tabeli `core_mainappuser`.



	id	last_login	first_name	last_name	username	password	mail
1		[NULL]	first	user	firstuser	test	firstuser@test.com
2	2	[NULL]	main	user	mainuser	pbkdf2_sha256$260000$4sJOFf3clke9JwJJHYaHPf$BT2GMnaicHWuqiRfUouHsQ3l0yVLS321HX/dkpkLhzo=	mainuser@test.com
3	5	[NULL]	asd	asdd	asduser	pbkdf2_sha256$260000$Eouq4zQ1H5mOLuGFjzmu6q$mwle8NrFkzzPRnbFIMs0r7c5GJF7kt+vXSwGjQycclc=	asduser@gmail.com
4	3	[NULL]	test21	test22	test2	pbkdf2_sha256$260000$5E4yuTtW4DS8ycxtGzmfZL$+3TvBnrUJOltC7CcSJ+7VVprx0uk/22Jva7zRY4NT18=	test2@test22.com
5	4	[NULL]	testing3	testing33	test3	pbkdf2_sha256$260000$UAVaDxH940YY6PE2MkaKLc$1MEtNG+o1NLwqXqHb4AxRvstPGrma61wQ1uXLOm8pUY=	test3@test33.com
6	6	[NULL]	Jarek	Mazur	igormazur	pbkdf2_sha256$260000$IaOUNulRXFS2yqR1Lj8JQc$xI8RS1sqyhuZiX2c9jUMqyOPnwb8OWIkCpkiYDVgfEE=	igor@gmail.com

Rysunek 3.25: Podgląd bazy danych po dokonaniu edycji.

Po zatwierdzeniu zmian dane w interfejsie graficznym zostały zaktualizowane automatycznie, bez konieczności ręcznego odświeżania strony. Finalny widok zaktualizowanego profilu został przedstawiony na rysunku 3.26.



Rysunek 3.26: Profil użytkownika po zapisaniu zaktualizowanych danych.

49:3745075328

3.5 Aspekty bezpieczeństwa aplikacji

Ze względu na charakter aplikacji, która została zaprojektowana z myślą o przechowywaniu i zarządzaniu między innymi kluczami kryptograficznymi w postaci sekwencji DNA, kwestie bezpieczeństwa stanowiły istotny element projektowy. Chociaż rozwiązanie nie było projektowane jako system produkcyjny, już na etapie prototypu podjęto działania mające na celu ochronę danych użytkowników.

W implementacji zastosowano następujące podstawowe mechanizmy zabezpieczające:

- uwierzytelnianie użytkowników z wykorzystaniem tokenów JWT, co pozwalało na kontrolę dostępu do chronionych zasobów interfejsu API,
- szyfrowanie haseł użytkowników przy użyciu algorytmu PBKDF2, co zapobiegało przechowywaniu danych uwierzytelniających w formie jawnej.

Należy jednak zauważyć, że w obecnej wersji aplikacja nie obsługuje szyfrowania transmisji danych (np. poprzez protokół HTTPS), co może prowadzić do potencjalnego przechwycenia wrażliwych informacji – takich jak zakodowane sekwencje DNA, przez osoby trzecie. Brak odpowiednich mechanizmów ochrony na poziomie sieciowym ogranicza zastosowanie systemu do celów demonstracyjnych.

Z tego względu projekt należy traktować jako prototyp, którego celem jest zaprezentowanie koncepcji zarządzania danymi o specyficznej strukturze i pochodzeniu (sekwencje DNA) w środowisku webowym oraz wykazanie umiejętności w zakresie integracji narzędzi backendowych i frontendowych. Aplikacja na obecnym etapie nie jest przeznaczona do wykorzystania produkcyjnego w środowisku rzeczywistym, gdzie wymagane są zaawansowane środki bezpieczeństwa.

Zagrożenie	**Możliwe zabezpieczenie**
Przechwycenie danych przez osoby trzecie	Zastosowanie protokołu HTTPS
Brak kontroli dostępu do API z obcych źródeł	Konfiguracja polityk CORS
Brak testów podatności aplikacji	Testy bezpieczeństwa np. OWASP ZAP
Nieaktualne zależności bibliotek	Regularne aktualizowanie zależności

Tabela 3.1: Zestawienie potencjalnych zagrożeń i możliwych zabezpieczeń

W kolejnych etapach rozwoju aplikacji wskazane byłoby uwzględnienie następujących usprawnień:

- implementacja pełnej obsługi protokołu HTTPS w celu zapewnienia bezpiecznej transmisji danych,

50:1092831901

- konfiguracja polityk CORS (Cross-Origin Resource Sharing), ograniczających dostęp do zasobów API wyłącznie z autoryzowanych źródeł,
- przeprowadzenie testów bezpieczeństwa (np. z użyciem narzędzi OWASP ZAP),
- wdrożenie systemu rejestracji i analizy logów oraz kontroli nad dostępem do przechowywanych danych,
- regularna aktualizacja zależności projektowych w celu eliminacji znanych podatności.

Tabela 3.1 zawiera zestawienie omawianych kwestii bezpieczeństwa. Uwzględnienie ograniczeń już na poziomie prototypu oraz świadomość ryzyk związanych z brakiem dodatkowych zabezpieczeń potwierdzają, że projekt został opracowany w sposób odpowiedzialny i może stanowić solidną bazę do dalszego rozwoju w kierunku aplikacji spełniającej standardy bezpieczeństwa stosowane w praktyce.

51:1534929572

Podsumowanie

Głównym celem niniejszej pracy było opracowanie aplikacji webowej, która integruje istniejący mechanizm kodowania danych w sekwencji DNA z przyjaznym dla użytkownika interfejsem graficznym. Cel ten został w pełni zrealizowany. Stworzony system umożliwia uwierzytelnionym użytkownikom konwersję danych cyfrowych na sekwencje nukleotydowe, stworzenie znacznika tagującego, zarządzanie wygenerowanymi zasobami oraz ich eksport w standardowym formacie.

Projekt z powodzeniem wykazał, że jest możliwe zbudowanie funkcjonalnego pomostu pomiędzy algorytmem bioinformatycznym a dostępnymi powszechnie technologiami webowymi. Realizacja pracy pozwoliła osiągnąc nie tylko postawione cele ale i pogłębić wiedzę z zakresu różnorodnych narzędzi programistycznych.

Zgodnie z założeniami przedstawionymi we wstępie, stworzone rozwiązanie stanowi prototyp, a jego głównym zadaniem była weryfikacja przyjętej koncepcji. Zaimplementowane podstawowe mechanizmy bezpieczeństwa, nie są wystarczające do zastosowań produkcyjnych.

Zrealizowanie podpunktów z ostatniego działu pracy, mogłoby przekształcić obecny prototyp w poważne narzędzie o potencjalnym zastosowaniu w badaniach naukowych lub archiwizacji danych.

52:5337196269

Spis rysunków

1.1 Ilustracja fragmentu nici DNA [2]. 6
1.2 Replikacja DNA [3]. 7
1.3 Cykl procedury PCR [4]. 9

2.1 Rysunek przedstawia wynik działania skryptu `bits.py`, który dokonuje podziału i wizualizuje strukturę nici DNA. 19
2.2 Schemat nici DNA. 20
2.3 Wynik działania skryptu `simple_tag.py`, który wizualizuje sposób stworzenia tagu DNA. 21

3.1 Ustawienia PostgreSQL w Dockerze oraz podgląd pliku `docker-compose.yml`. 27
3.2 Odpowiedź z `/api/token/` – token dostępowy i odświeżający. 33
3.3 Odpowiedź na zapytanie GET do `/api/token/user/`. 33
3.4 Odpowiedź z `/api/token/refresh/` - nowy token dostępowy. 34
3.5 Odpowiedź GET z `/simple-tag/` – lista sekwencji DNA. 34
3.6 Lista tagów zalogowanego użytkownika – `/simple-tag/my-tags/`. 35
3.7 Lista wygenerowanych sekwencji kodujących bity – `/generate-bits/my-bits/`. 36
3.8 Podgląd tagów w bazie. 36
3.9 Odpowiedzi po wykonaniu żądań DELETE na adresy `/simple-tag/my-tags/21/` oraz `/generate-bits/my-bits/16/`. 37
3.10 Podgląd bazy po prawidłowym wykonaniu operacji DELETE. 37
3.11 Podgląd pierwszych widoków. 39
3.12 Walidacja danych w oknie rejestracji. 40
3.13 Możliwy podgląd wpisanego hasła, po wciśnięciu guzika w polu *Password*. 40
3.14 Panel główny. 41
3.15 Okno dialogowe, otwierane na środku ekranu. 43
3.16 Okno dialogowe oczekujące na dane wejściowe od użytkownika. 43
3.17 Dodatkowe przyciski ułatwiające, tzw. QoL (ang. quality of life). 44
3.18 Wysunięte boczne menu nawigacyjne. 45

53:2616032515

3.19 Historia poprzednio wygenerowanych tagów. 45
3.20 Szczegóły pojedynczego taga. 46
3.21 Szczegóły dla sekwencji kodującej bity wprowadzone przez użytkownika. . 47
3.22 Automatycznie wygenerowany plik w formacie `.json` 47
3.23 Profil użytkownika przed aktualizacją danych. 48
3.24 Podgląd bazy danych przed edycją profilu. 49
3.25 Podgląd bazy danych po dokonaniu edycji. 49
3.26 Profil użytkownika po zapisaniu zaktualizowanych danych. 49

54:4228394272

Listings

2.1 Przykładowy JSON 24
3.1 Ustawienia Simple JWT. 28
3.2 Modele z `models.py`. 29
3.3 Przykładowy Widok z `views.py`, obsługujący żądania sieciowe z URLu `/simple-tag/`. 30
3.4 Plik `serializers.py`. 31
3.5 Konfiguracja ścieżek z `urls.py`. 32
3.6 Podgląd pliku `App.js`, w którym znajdują się ścieżki. 38
3.7 Funkcja *handleSubmit*, odpowiada za wywołanie funkcji *authService.login* z danymi pobranymi z forumlarza logowania. W przypadku powodzenia, kieruje ona użytkownika do widoku (URL) `/main`. 39
3.8 Fragment kodu z komponentu odpowiadającego za widok ekranu głównego z rysunku 3.14. 41
3.9 Funkcja **handleSimpleTag** 43
3.10 Metoda realizująca żądanie do API w celu wygenerowania taga 44
3.11 Fragment kodu odpowiedzialnego za pobieranie danych w formacie JSON. 48

55:8081039724

Bibliografia

[1] N. Goldman, P. Bertone, S. Chen, *et al.*, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, nr. 7435, str. 77–80, 2013. DOI: 10.1038/nature11875.

[2] D. Adryan, "Dna - nośnik informacji genetycznej," 2022. [Online]. Dostęp: https://zpe.gov.pl/a/dna---nosnik-informacji-genetycznej/D18yvChME [Dostęp: 2022-03-13].

[3] A. Mandal, "What is DNA Polymerase?" 2022. [Online]. Dostęp: https://www.news-medical.net/life-sciences/What-is-DNA-Polymerase.aspx [Dostęp: 2022-09-28].

[4] CliniSciences, "Conventional PCR Products," 2025. [Online]. Dostęp: https://www.clinisciences.com/es/comprar/cat-conventional-pcr-3473.html [Dostęp: 2025-03-11].

[5] L. C. Meiser, B. H. Nguyen, Y.-J. Chen, *et al.*, "Synthetic DNA applications in information technology," *Nature Communications*, vol. 13, nr. 1, str. 352, 2022. DOI: 10.1038/s41467-021-27846-9.

[6] Y. Erlich and D. Zielinski, "DNA Fountain enables a robust and efficient storage architecture," *Science*, vol. 355, nr. 6328, str. 950–954, 2017. DOI: 10.1126/science.aaj2038.

[7] L. Ceze, J. Nivala, and K. Strauss, "Molecular digital data storage using DNA," *Nature Reviews Genetics*, vol. 20, nr. 8, str. 456–466, 2019. DOI: 10.1038/s41576-019-0125-3.

[8] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, nr. 8, str. 2552–2555, 2015. DOI: 10.1002/anie.201411378.

[9] ADNAS, "DNA Tagging and the Internet of Things," 2018. [Online]. Dostęp: https://web.archive.org/web/20181005214215/http://adnas.com/2018/06/01/dna-tagging-and-the-internet-of-things/ [Dostęp: 2022-09-29].

56:9130718797

[10] P. Pastore, "DNA tagged cotton will change the way you shop," 2017. [Online]. Dostęp: https://crfashionbook.com/dna-tagged-cotton [Dostęp: 2022-09-29].

[11] J. Warner, "No funny business when cartoons are signed with DNA," 1995. [Online]. Dostęp: https://www.newscientist.com/article/mg14619713-300-no-funny-business-when-cartoons-are-signed-with-dna/ [Dostęp: 2022-09-29].

[12] A. D. Sciences, "DNA Marking and Authentication: A unique, secure anti-counterfeiting program for the electronics industry," 2011. [Online]. Dostęp: https://www.dla.mil/Portals/104/Documents/LandAndMaritime/V/VA/PSMC/Nov11/LM_DNAMarkingAndAuthentication_151030.pdf [Dostęp: 2025-07-16].

[13] SelectaDNA, "SelectaDNA - Vehicle Marking," 2025. [Online]. Dostęp: https://www.selectadna.co.uk/vehicle-marking [Dostęp: 2025-07-16].

[14] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, nr. 6102, 2012. DOI: 10.1126/science.1226355.

[15] M. Miśkiewicz, A. Kuzdraliński, D. Rusinek, and B. Księżopolski, "DNA based cryptographic key storage system with a simple and automated method of primers selection," *Selected Topics in Applied Computer Science*, vol. 5, nr. 1, str. 1–8, 2021.

[16] M. Miśkiewicz, "Primers small set dataset," 2022. GitHub, [Online]. Dostęp: https://github.com/miskiewiczm/bits_concept/blob/main/primers_small_set.txt [Dostęp: 2022-05-31].

[17] M. Miśkiewicz, "Koncepcja tagowania za pomocą DNA," 2022. GitHub, [Online]. Dostęp: https://github.com/miskiewiczm/bits_concept/blob/main/concept.md [Dostęp: 2022-05-31].

[18] L. Gupta, "What is REST?" 2022. [Online]. Dostęp: https://restfulapi.net/ [Dostęp: 2022-09-27].

[19] T. Fredrich, "Using HTTP Methods for RESTful Services," 2022. [Online]. Dostęp: https://www.restapitutorial.com/lessons/httpmethods.html [Dostęp: 2022-09-27].

57:2503321373